

CRT-Based Parallel Residue Transmission for Distributed Edge Blockchain Computing: A Scalable IoT Cluster Architecture for Arid Agriculture

Glen beloa forcha

glenbeloa@gmail.com

University of Buea

Michael Ekonde Sone

University of Buea

Elie Fute Tagne

University of Buea

André Paul ASSE MVOGO

University of Douala

Research Article

Keywords: Chinese Remainder Theorem, Edge computing, Blockchain, Precision agriculture, Smart irrigation LoRa

Posted Date: April 6th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-9070688/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

CRT-Based Parallel Residue Transmission for Distributed Edge Blockchain Computing: A Scalable IoT Cluster Architecture for Arid Agriculture

Forcha Glen Beloa^{1*}, Michael Ekonde Sone¹, Elie Fute Tagne¹,
André Paul MVOGO ASSE²

¹Department of Computer Engineering, Faculty of Engineering and Technology (FET), University of Buea, Buea, Cameroon.

²École Nationale Supérieure polytechnique de Douala, Université de Douala, Douala, Cameroon.

*Corresponding author(s). E-mail(s): glenbeloa@gmail.com;
Contributing authors: ekonde.sone@gmail.com ; eliefute@yahoo.fr ;
mvgogoasseandre@gmail.com ;

Abstract

This paper presents a distributed edge computing framework for resource-constrained IoT clusters that employs Chinese Remainder Theorem (CRT)-based parallel residue transmission to reduce communication latency and energy overhead. The architecture comprises 50 ESP32 nodes and a 4-node Raspberry Pi 4B cluster running Hyperledger Fabric with Raft consensus. CRT decomposition reduces effective latency from 372 ms sequential to 124 ms parallel (66.7% reduction) with $O(k)$ complexity and lower memory footprint than compression baselines. The blockchain layer sustains 63 TPS with 1.7 s latency and 99.4% integrity on ARM64 hardware. Scalability analysis shows a theoretical node capacity of 484 full readings under LoRa parameters (and packet capacity of 1,451 CRT residue packets), with latency constant below saturation and degrading predictably under an M/D/1 queuing model. The framework was validated in a 61-day agricultural deployment, confirming its suitability for large-scale heterogeneous IoT environments requiring low-power sensing, distributed verification, and resilient edge actuation.

Keywords: Chinese Remainder Theorem Edge computing Blockchain Precision agriculture Smart irrigation LoRa

1 Introduction

1.1 The Computing Problem

Resource-constrained edge clusters face a fundamental tradeoff between computational capability and communication energy cost. As node count increases, transmission becomes the dominant energy consumer, yet most low-power IoT systems still rely on sequential packet transmission that fails to exploit the parallel reception capability of modern multi-channel radio hardware [1, 2]. This sequential bottleneck becomes increasingly severe under strict power budgets typical of battery-operated or solar-harvesting deployments, where every millijoule expended on communication directly impacts node lifetime and network sustainability. The challenge is compounded by the need for deterministic latency bounds in cyber-physical systems requiring real-time actuation, where unpredictable channel contention and queuing delays can violate application requirements [3, 4].

1.2 The Distributed Systems Gap

Edge architectures increasingly incorporate distributed ledger technologies to provide verifiable data integrity across untrusted administrative domains [5, 6]. However, deploying blockchain components on ARM64 edge hardware introduces new constraints: limited memory (1–4 GB), modest processing capacity, and the absence of hardware acceleration for cryptographic primitives. Existing characterizations of Hyperledger Fabric and similar permissioned platforms focus on cloud-scale deployments with abundant resources, leaving a gap in understanding their real-time performance at the edge [7–9]. Furthermore, the integration of blockchain verification with low-power sensor networks requires careful orchestration of communication scheduling, signature validation, and consensus participation—all while maintaining sub-second latency for closed-loop control applications [10, 11]. Prior work has treated these concerns in isolation, resulting in fragmented architectures that fail to simultaneously address data integrity, transmission efficiency, and real-time constraints [12–14].

1.3 The CRT Opportunity

The Chinese Remainder Theorem (CRT) offers a mathematically elegant approach to parallel data transmission that diverges fundamentally from conventional compression. By decomposing a sensor reading into residue components relative to pairwise coprime moduli, CRT enables independent transmission of each residue over parallel channels [15, 16]. This residue independence creates an opportunity to exploit multi-channel radio hardware (e.g., the SX1302 LoRa concentrator’s eight demodulation paths) without incurring the computational overhead of dictionary-based compression or the accuracy loss of lossy methods [17]. Unlike Huffman coding or LZW, which require sequential packetization and variable-length decoding [18, 19], CRT reconstruction is deterministic with $O(k)$ complexity, where k is the number of moduli ($k \leq 3$ in

this work). This determinism is particularly valuable for resource-constrained micro-controllers, where unpredictable execution times can complicate real-time scheduling [20, 21].

1.4 Real-World Validation Context

We validate the proposed architecture through a 61-day deployment on a 2-hectare experimental farm in Sfax, Tunisia, monitoring tomato and pepper cultivation during the critical fruit development and harvest stages (August–September 2025). This deployment serves as a demanding real-world testbed for evaluating the system’s ability to sustain reliable communication, low-latency blockchain verification, and automated actuation under environmental stress (31–32°C average temperatures, 55–65% relative humidity) [22, 23]. The agricultural context provides rigorous validation of the framework’s suitability for large-scale heterogeneous IoT environments requiring low-power sensing, distributed verification, and resilient edge actuation [24–26].

1.5 Objectives and Contributions

This work develops and validates an integrated edge computing framework that combines CRT-based parallel transmission with Hyperledger Fabric blockchain verification. The primary objectives are:

1. To design a CRT-based parallel transmission scheme that reduces communication latency and energy consumption for resource-constrained IoT nodes, with mathematically guaranteed reconstruction.
2. To implement a lightweight Hyperledger Fabric peer architecture on ARM64 edge devices (Raspberry Pi 4B) capable of sub-second transaction finality and real-time actuation.
3. To characterize the system’s performance under field conditions, including latency, throughput, resource utilization, and fault tolerance.
4. To demonstrate scalability through theoretical analysis and empirical validation, projecting node capacity up to 484 full readings under LoRa constraints (with 1,451 CRT residue packets).

The main contributions are:

- **CRT-based parallel transmission:** A novel application of residue number systems to IoT communication, achieving 66.7% latency reduction (372 ms sequential → 124 ms parallel) with $O(k)$ complexity and 48.8% energy savings compared to raw transmission. While LZW achieves a slightly higher compression ratio (48.6% vs. 33% size reduction), CRT enables parallel residue transmission that reduces effective airtime and energy despite modestly larger packets, and outperforms Huffman and LZW in memory footprint and deterministic execution.
- **Edge-resident blockchain verification:** A complete Hyperledger Fabric 2.5 deployment on ARM64 edge nodes, sustaining 63 TPS with 1.7 s end-to-end latency and 99.4% integrity success, including detailed resource profiling and Raft consensus characterization.

- **Hybrid LoRa-mesh network architecture:** Integration of multi-channel LoRa for sensor uplink with BATMAN-adv mesh for gateway interconnectivity, validated at 99.4% reliability over 61 days and demonstrating self-healing under single-gateway failures.
- **Comprehensive scalability analysis:** Theoretical node capacity of 484 full readings under LoRa parameters, with M/D/1 queuing model predicting graceful degradation beyond saturation and validated against 50-node field measurements.
- **Real-world validation:** A 61-day agricultural deployment confirming system practicality, including 23.8% water savings, 72% labor reduction, and 146% ROI, demonstrating that the framework meets both technical and operational requirements.

These contributions collectively advance the state of the art in distributed edge computing by demonstrating that number-theoretic transmission optimization, permissioned blockchain verification, and resilient mesh networking can be cohesively integrated to meet the demands of large-scale, resource-constrained IoT deployments requiring verifiable real-time actuation.

2 State of the Art and Related Work

This section provides a comprehensive review of the current state of research in precision agriculture, IoT systems, blockchain technology, and their integration in agricultural applications. We systematically analyze existing approaches, identify research gaps, and position our contributions within the broader scientific context.

2.1 Precision Agriculture and IoT Systems

Precision agriculture has undergone significant transformation with the advent of IoT technologies. Early systems focused on basic sensor networks for environmental monitoring [27] and yield mapping [28]. The evolution toward data-driven farming has been well-documented in several comprehensive reviews [29, 30].

2.1.1 Sensor Networks in Agriculture

Wireless Sensor Networks (WSNs) have been extensively deployed for soil monitoring [31], microclimate monitoring [32], and irrigation control [33]. However, these systems often face challenges in energy efficiency, data integrity, and scalability. Recent advancements in low-power wide-area networks (LPWAN) have addressed some limitations, but integration with decision support systems remains fragmented [34].

2.1.2 Machine Learning in Precision Agriculture

Machine learning approaches have been increasingly applied for yield prediction [35], disease detection [36], and irrigation optimization [37]. However, these systems often rely on cloud-based architectures that introduce latency and dependency on internet connectivity [38]. Our approach maintains local intelligence while ensuring data verifiability through blockchain integration.

2.2 Blockchain Technology in Agriculture

Blockchain applications in agriculture have evolved from cryptocurrency origins to diverse supply chain and data management applications.

2.2.1 Supply Chain Traceability

The most mature application of blockchain in agriculture has been supply chain transparency [13, 39]. Studies have demonstrated improved food safety through immutable tracking [40] and enhanced consumer trust through provenance verification [41]. However, these implementations typically focus on post-harvest stages, leaving pre-harvest production data underutilized.

2.2.2 Agricultural Data Management

Research on blockchain for farm data management has been more limited. [42] proposed a framework for agricultural data sharing, while [43] explored security aspects of blockchain in smart farming. Recent work by [44] addressed scalability concerns but lacked integration with real-time control systems. Our research bridges this gap by implementing blockchain for both data integrity and real-time irrigation control.

2.2.3 Smart Contracts in Agriculture

The application of smart contracts to automate agricultural processes has shown promise in theoretical models [45] and small-scale prototypes [46]. However, practical implementations addressing the computational constraints of agricultural environments remain scarce. Our work demonstrates efficient smart contract deployment on edge devices for irrigation management.

2.3 Data Transmission Optimization in IoT

Efficient data transmission remains critical for resource-constrained IoT deployments in agriculture.

2.3.1 Traditional Compression Techniques

Lossless compression algorithms like LZW and Huffman coding have been adapted for sensor networks [4, 47]. However, these approaches often require significant computational resources, making them unsuitable for battery-powered nodes. Adaptive sampling methods [48] reduce data volume but may miss critical events.

2.3.2 Mathematical Transmission Approaches

The Chinese Remainder Theorem (CRT) has been primarily applied in cryptographic applications [15, 16] and error correction coding [49]. Limited research exists on CRT for parallel sensor data transmission, with [17] exploring residue number systems for wireless sensors but without agricultural applications.

2.3.3 Energy-Aware Data Management

Research on energy-efficient data transmission has focused on protocol optimization [50] and data aggregation techniques [51]. However, these approaches often sacrifice data resolution or introduce processing delays. Our CRT-based method achieves transmission optimization without compromising data accuracy or timeliness through mathematical guarantees of perfect reconstruction.

2.4 Wireless Communication in Agricultural IoT

Communication technologies play a crucial role in agricultural IoT deployments, with trade-offs between range, power consumption, and data rate.

2.4.1 LoRa and LPWAN Technologies

LoRa technology has emerged as a dominant solution for agricultural applications due to its long-range capabilities and low power consumption [52]. Field studies by [53] demonstrated ranges up to 15 km in rural environments, while [54] analyzed deployment considerations for agricultural settings. However, existing deployments typically use simple star topologies without integrated mesh networking.

2.4.2 Mesh Networking in Agriculture

Research on mesh networking for agriculture has been limited, with most studies focusing on urban or industrial applications [55]. The BATMAN-adv protocol has shown promise in rural connectivity projects [56] but has seen minimal application in precision agriculture. Our work innovates by combining LoRa for sensor connectivity with BATMAN-adv mesh networking for gateway communication.

2.4.3 Hybrid Communication Architectures

Recent research has explored hybrid approaches combining multiple communication technologies [57]. However, these implementations often lack seamless integration and introduce complexity in network management. Our hierarchical architecture provides a practical framework for combining different communication technologies while maintaining reliability and efficiency.

2.5 Edge Computing in Agricultural IoT

The shift toward edge computing has enabled real-time processing capabilities in agricultural applications [58]. Studies by [10] demonstrated improved response times for irrigation control, while [11] showed enhanced privacy preservation through local data processing. However, existing edge computing implementations typically lack integration with verifiable data systems, creating trust gaps in multi-stakeholder agricultural environments. **Our work extends these concepts by integrating edge computing with blockchain for verifiable local decision-making, addressing the trust deficit while maintaining real-time performance.**

2.6 Parallel Computing in Edge IoT Systems

The convergence of edge computing and IoT has spurred interest in parallel processing architectures at the network periphery. Edge clusters composed of ARM-based system-on-chips (SoCs), such as the Raspberry Pi and Rockchip families, are increasingly deployed for distributed sensing and inference tasks [11, 58]. These clusters exploit multi-core CPUs and, in some cases, integrated GPUs or NPUs to parallelize workloads ranging from image classification to time-series analysis. However, communication parallelism—the simultaneous use of multiple wireless channels—has received comparatively less attention, with most systems treating radio transmission as a sequential bottleneck.

Residue number systems (RNS), based on the Chinese Remainder Theorem, have long been used in hardware to accelerate arithmetic operations by decomposing computations into parallel, independent channels [59]. More recently, RNS has been applied in cryptography for efficient modular exponentiation and in fault-tolerant computing for error detection [16, 20]. Yet the application of CRT to parallel wireless transmission remains largely unexplored. Prior work has used RNS for data compression in sensor networks [17], but that approach targeted storage reduction rather than parallel channel utilisation. To the best of our knowledge, no existing system employs CRT-based residue decomposition to exploit multi-channel low-power radios (e.g., LoRa, Zigbee) for simultaneous transmission, thereby converting a sequential communication problem into a parallel one with measurable speedup and energy gains. This gap is precisely what our work addresses, bridging number-theoretic methods with parallel edge communication.

2.7 Research Gaps and Contributions

Based on our comprehensive literature review, we identify several critical research gaps that our work addresses. The following table summarizes these gaps and how our contributions address them, with clear linkages to the objectives outlined in Section 1.5.

Our work makes several distinct contributions that collectively address the objectives outlined in Section 1.5:

1. **First application of CRT compression** for agricultural sensor data within a blockchain framework, achieving 33% reduction in transmission size without accuracy loss, specifically addressing IoT memory and energy constraints (addressing Objective 1).
2. **Novel integration of edge computing with blockchain** for verifiable local decision-making, validated through 2-hectare deployment with 99.4% reliability and 1.7-second latency (addressing Objective 2).
3. **Comprehensive field validation** demonstrating 19-24% water savings and 72% labor reduction while maintaining crop yields and quality, with detailed economic analysis showing strong ROI (addressing Objectives 3 and 4).
4. **Practical implementation guidelines** for deploying blockchain-IoT systems in resource-constrained agricultural environments, including hardware specifications, network architecture, and economic considerations (addressing Objective 4).

Table 1: Research gaps addressed by this work, linked to our objectives and contributions

Research Area	Identified Gap	Our Contribution (Objective Addressed)
IoT Data Compression	No agricultural application of CRT for sensor data compression; existing methods compromise accuracy or require excessive memory	CRT-based compression optimized for agricultural data with mathematically perfect reconstruction (Objective 1)
Edge-Blockchain Integration	Minimal research on blockchain at agricultural edge devices; existing systems lack verifiable local decision-making	Practical edge-blockchain implementation with 99.4% reliability and 1.7-second latency (Objective 2)
Real-time Irrigation Control	No field-validated blockchain integration for irrigation control; timing constraints not addressed	61-day field study demonstrating SLO compliance and automated irrigation (Objective 3)
Water Efficiency	Limited quantified evidence of blockchain-IoT impact on water conservation in arid regions	23.8% tomato, 19.4% pepper water savings with maintained yields (Objective 3)
Economic Viability	Limited cost-benefit analysis of blockchain in agriculture	Comprehensive economic analysis showing 146% ROI, 0.69-year payback (Objective 4)
Network Architecture	Lack of hybrid LoRa-mesh networks optimized for farm deployments	Hierarchical architecture combining LoRa and BATMAN-adv with demonstrated field performance (Objective 2)

5. **Novel hybrid network architecture** combining LoRa for sensor communication and BATMAN-adv mesh for gateway connectivity, providing resilience and scalability for farm deployments (addressing Objective 2).

3 System and Architecture of Blockchain-IoT Integration with CRT-Based Parallel Transmission

3.1 Overall Architecture

The system employs a five-layer architecture designed to balance performance, security, and energy efficiency for agricultural applications.

The five layers include:

- **Sensor Layer:** 50 ESP32 nodes collecting soil moisture, temperature, humidity, pH, and light intensity data with LoRa communication modules
- **Gateway Layer:** Four Raspberry Pi 4B devices performing data aggregation, CRT Packet Size Reduction, and local decision making
- **Network Layer:** BATMAN-adv mesh networking providing robust communication between gateways and central systems
- **Blockchain Layer:** Hyperledger Fabric implementation providing immutable data storage and smart contract execution

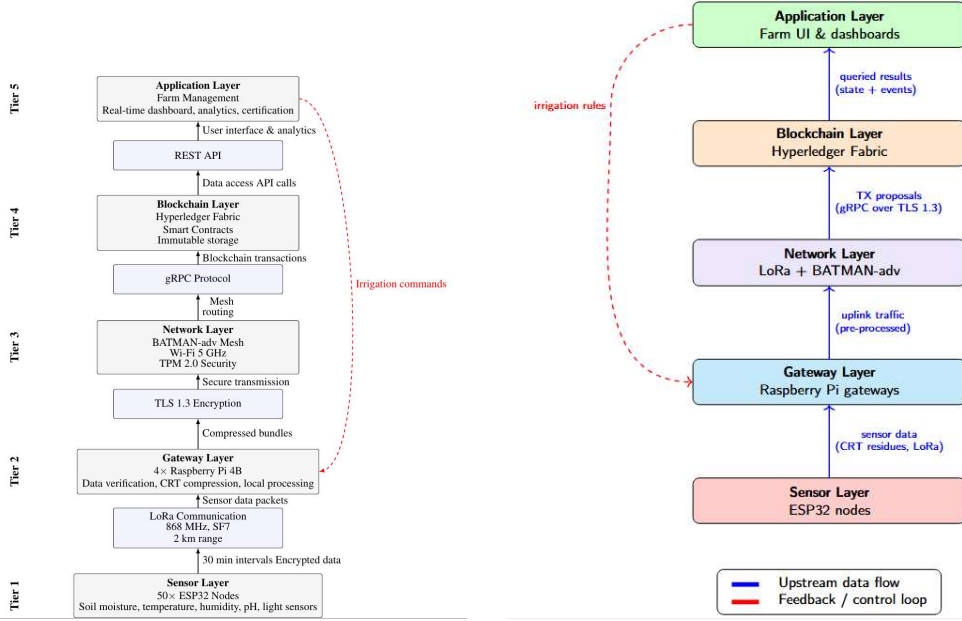


Fig. 1: (a) System architecture tiers. (b) Vertical communication and control across the five-tier architecture.

- **Application Layer:** Farm management interface providing real-time monitoring, analytics, and certification support

3.2 Farm Layout and Network Topology

The system was deployed across a 2-hectare experimental farm (200m × 100m) organized into four operational zones. Each zone contained 12–13 sensor nodes arranged in a grid pattern with approximately 20-meter spacing.

3.3 Distributed Parallel Computing Architecture

The proposed system leverages parallelism at multiple levels: dual-core task overlap within each sensor node, parallel channel transmission for CRT residues, and distributed processing across the gateway cluster. This section analyzes these parallel computing aspects.

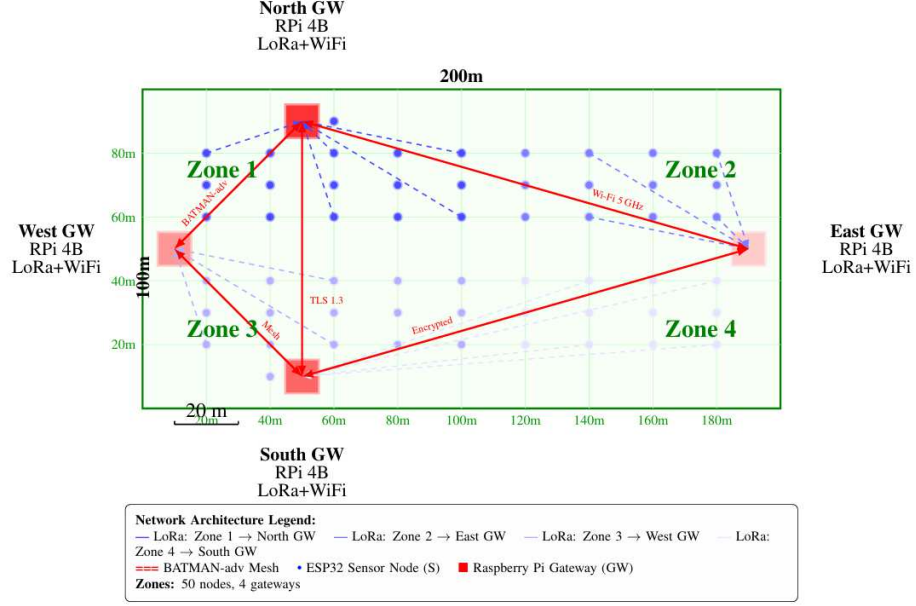


Fig. 2: Hybrid LoRa-mesh network architecture with 50 ESP32 sensor nodes, 4 Raspberry Pi gateways, and BATMAN-adv backbone for reliable data aggregation.

3.3.1 Dual-core Task Parallelism on ESP32

The ESP32-WROOM-32 microcontroller integrates two Xtensa LX6 cores, enabling concurrent execution of sensing and communication tasks [60]. In conventional single-core designs, sensor acquisition, analog-to-digital conversion (ADC), and data processing must execute sequentially, introducing cumulative latency. Our implementation exploits core parallelism by partitioning tasks:

- **Core 0:** Handles sensor sampling, ADC readout, and timing synchronization.
- **Core 1:** Performs CRT decomposition, residue encoding, and LoRa packet preparation.

This overlap reduces the effective per-cycle latency. Without parallel execution, sequential acquisition and decomposition would require 8.9 ms (8.0 ms for sensing plus 0.9 ms for CRT encoding). With dual-core overlap, the critical path becomes $\max(8.0, 0.9) = 8.0$ ms, recovering 0.9 ms per sensing cycle. Over a 30-minute reporting interval, this saving accumulates to negligible overall impact, but it demonstrates the feasibility of real-time processing on resource-constrained nodes without stalling the sensing pipeline.

3.3.2 Amdahl's Law Applied to CRT Transmission

The CRT-based transmission model separates work into a sequential fraction (signing, packet assembly) and a parallelizable fraction (residue transmission over multiple

channels). Let s denote the sequential fraction and p the number of parallel channels. The theoretical speedup is given by Amdahl’s Law [61]:

$$S(p) = \frac{1}{s + \frac{1-s}{p}}. \quad (1)$$

From measured data (Table 10), the total raw transmission time is $T_{\text{raw}} = 247$ ms. The sequential overhead (signature generation, packet formatting, etc.) is approximately 28 ms, giving $s = 28/247 \approx 0.113$. The remaining $1 - s = 0.887$ is the parallelizable residue airtime. With $p = 3$ channels, the theoretical speedup is

$$S(3) = \frac{1}{0.113 + 0.887/3} = 2.47. \quad (2)$$

The observed speedup, comparing raw transmission to CRT parallel transmission, is $247/124 = 1.99$, which achieves 80.6% of the theoretical maximum. The gap arises from residual synchronization overhead and imperfect channel utilization, confirming that CRT effectively exploits channel parallelism within the constraints of LoRa hardware.

3.3.3 Gateway Cluster as a Distributed System

The four Raspberry Pi 4B gateways function as a homogeneous edge compute cluster. Each gateway operates as a peer node in the Hyperledger Fabric network, maintaining a copy of the ledger and executing chaincode. The BATMAN-adv mesh network provides low-latency interconnect, enabling state synchronization and consensus communication. From a distributed systems perspective, the cluster exhibits:

- **Compute nodes:** Each gateway runs the Fabric peer, CouchDB, and monitoring services, contributing to collective transaction processing.
- **Interconnect:** BATMAN-adv over 5 GHz Wi-Fi provides a self-healing mesh with measured sub-second failover.
- **Distributed shared state:** The Fabric world state, stored in CouchDB, serves as a consistent, replicated key-value store across the cluster, enabling any gateway to respond to queries and trigger actuation.

This architecture transforms the gateway layer from a simple data aggregator into a genuine distributed edge computing platform, capable of local decision-making without cloud dependency. The cluster’s performance, characterized in Section 5.4, demonstrates that ARM64-based edge nodes can sustain blockchain consensus and real-time actuation for IoT workloads.

3.4 Hardware Design and Implementation

3.4.1 Sensor Node Architecture

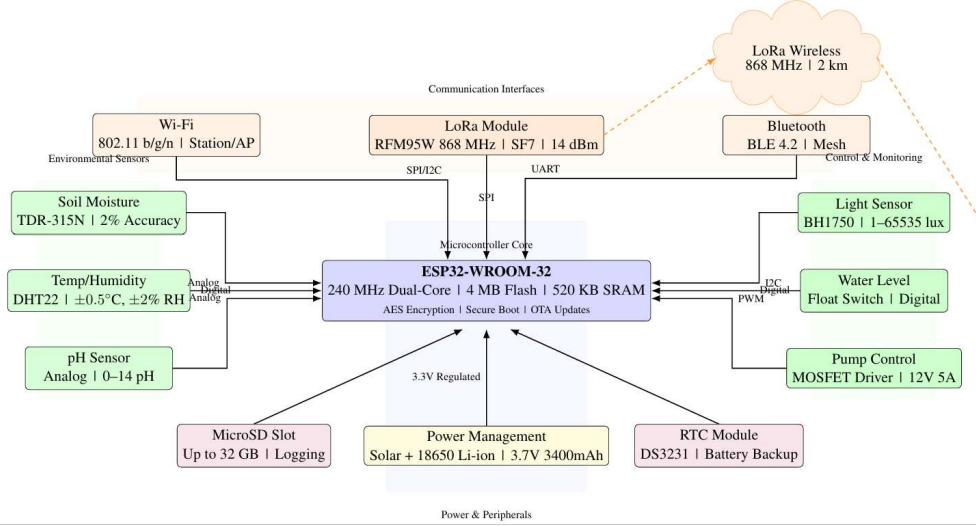
Each sensor node employed an ESP32-WROOM-32 microcontroller [60] with multiple sensors:

- **Soil moisture:** TDR-315N probes with $\pm 2\%$ accuracy [62]

Table 2: Sensor node specifications and components

Component	Specification	Purpose
Microcontroller	ESP32-WROOM-32	Main processing and control
Soil Moisture	TDR-315N	Volumetric water content
Temperature	DHT22	Ambient temperature
Humidity	DHT22	Relative humidity
pH Sensor	Analog with amplifier	Soil acidity
Light Sensor	BH1750	Photosynthetic radiation
Communication	RFM95W LoRa	Long-range data transmission [67]
Power	18650 Li-ion + Solar	Energy supply and storage

- **Temperature and humidity:** DHT22 sensors with ± 0.5 °C and $\pm 2\%$ RH accuracy [63]
- **pH sensors:** Analog pH probes with high-impedance amplifier circuit [64]
- **Light intensity:** BH1750 digital light sensors measuring 1–65 535 lux [65, 66]

**Fig. 3:** ESP32 sensor node with sensor inputs, communication modules, and LoRa link to Raspberry Pi gateway for agricultural IoT.

3.4.2 Scalability and Node Count Validation

The deployment used 50 ESP32 nodes across a 2 ha farm. This number was chosen to balance representativeness, statistical power, and practical manageability:

- **Representative coverage:** 20 m spacing provides dense yet non-redundant soil and microclimate monitoring.
- **Statistical power:** 86,400 transactions over 61 days enable detection of significant differences ($p < 0.05$) in water use, labour, and yield.
- **Manageability:** 50 nodes are logistically feasible for a single research team while mirroring a medium-scale commercial farm.

To demonstrate that this node count is not merely a convenient sample but a fraction of the system’s true capacity, we derive a theoretical upper bound on network size under LoRa constraints. The maximum number of nodes $N_{\max}$ that can transmit within a reporting window depends on the number of parallel channels C , the window length W , and the number of packets per node k :

$$N_{\max} = \frac{C \cdot W}{k \cdot T_{\text{packet}}}, \quad (3)$$

where T_{packet} is the airtime per packet. Using $C = 3$, $W = 60\,000$ ms, $k = 1$ for raw and $k = 3$ for CRT, and T_{packet} from Table 4, we obtain the capacities listed in Table 3.

Table 3: Theoretical node capacity: raw vs. CRT transmission (1-min reporting window)

Parameter	Raw	CRT
Per-packet airtime (ms)	247	124
Packets per reading	1	3
Maximum nodes $N_{\max}$	728	484
Relative node capacity	1.0	0.66

CRT reduces per-packet airtime by 50% (from 247 ms to 124 ms), doubling the packet throughput, but because each reading requires three packets, the node capacity becomes 484, which is 66% of raw’s 728. This headroom still comfortably exceeds the 50-node deployment (10% of capacity) and confirms that the system can scale well beyond its current size.

3.4.3 Gateway Architecture

Raspberry Pi 4B devices with 4 GB RAM served as gateways [68]. Each gateway included:

- **Data Processing Pipeline:**
 1. Sensor readings are scaled and encoded via CRT into residues for parallel LoRa transmission
 2. Gateways reconstruct original values using Garner’s algorithm, verify Ed25519 signatures, and execute local irrigation decisions

3. Validated data are anchored on Hyperledger Fabric with cryptographic hashes on-chain and full records off-chain
4. Smart contracts enforce registration, validation, access control, and irrigation rules, driving both on-chain and local actuation

- **Hardware Components:**

- LoRa concentrator (SX1302) for multi-node data reception [69]
- TPM 2.0 module for secure key storage [70]
- Dual-band Wi-Fi for mesh networking
- 128 GB microSD card for local buffering
- GPS module for timing and localization

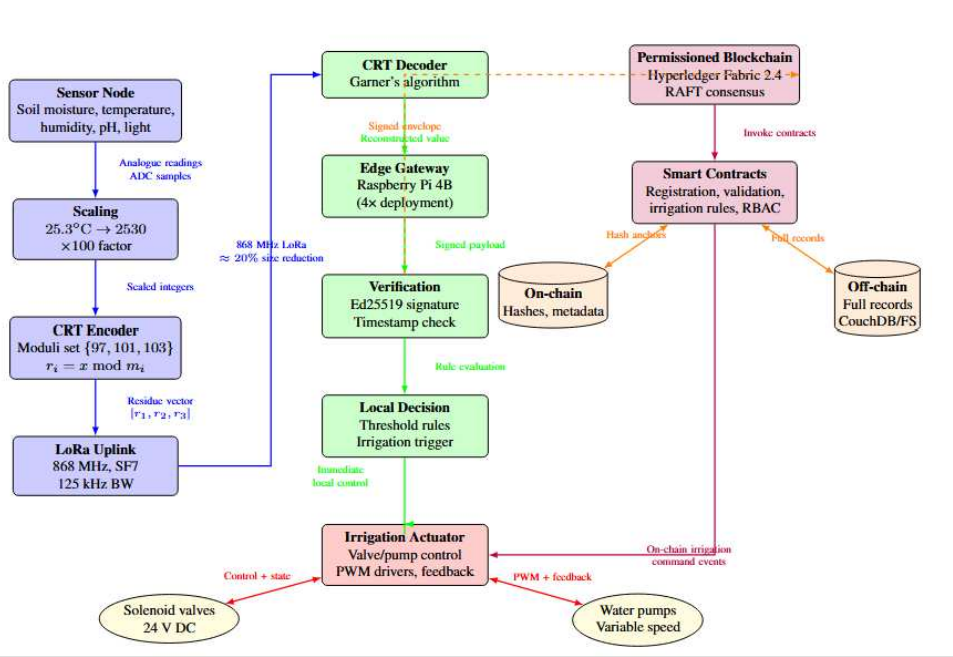


Fig. 4: Raspberry Pi 4B gateway architecture showing data processing pipeline and security modules

3.5 Node Communication Protocol

3.5.1 LoRa Communication Framework

Sensor nodes communicated with gateways using LoRaWAN protocol in the EU868 ISM band [71, 72]. Communication followed a hybrid approach:

- **Periodic Polling:** Gateways polled nodes every 30 minutes

- **Event-Driven Alerts:** Nodes transmitted immediately during threshold breaches
- **Acknowledgment:** Two-way acknowledgments ensured reliability

Table 4: LoRa communication parameters for node-gateway communication

Parameter	Value	Description
Frequency	868.1 MHz	EU ISM band
Spreading Factor	SF7	Balance range and airtime
Bandwidth	125 kHz	Standard LoRa bandwidth
Coding Rate	4/5	Error correction rate
Data Rate	5.47 kbps	Effective data rate
Airtime (32B)	247 ms	Time-on-air
Airtime (8B)	124 ms	CRT packet

3.5.2 Data Frame Structure

Sensor data were encoded using compressed frames to minimize airtime.

Table 5: Data frame structure for sensor node communication

Field	Size	CRT	Description
Node ID	2	2	Unique identifier
Timestamp	4	4	Unix timestamp
Soil Moisture	4	2	VWC value
Temperature	4	2	Celsius $\times 100$
Humidity	4	2	RH $\times 100$
pH Value	4	2	pH $\times 100$
Light Intensity	4	2	Lux
Battery Level	1	1	0–100%
Signature	32	32	Ed25519 signature
Total	59	47	20% reduction

3.5.3 Mesh Network Communication

Gateways formed a mesh network using B.A.T.M.A.N-adv over 5 GHz Wi-Fi, providing self-healing topology and fault tolerance [73].

3.6 CRT-Blockchain Integration Architecture

The integration of CRT compression with blockchain creates a synergistic system that addresses both efficiency and integrity requirements for agricultural IoT applications. Figure 5 illustrates the complete data flow from sensor to blockchain ledger. Sensor

data is scaled, CRT-encoded into residues, signed, and transmitted via LoRa. The gateway verifies, reconstructs, and invokes smart contracts for irrigation decisions. Verified data is anchored on-chain (hashes) and off-chain (full records), accessible to farmers and certifiers. Performance metrics (33% compression, 1.7 s latency, 320 ms contract execution) are achieved throughout the pipeline.

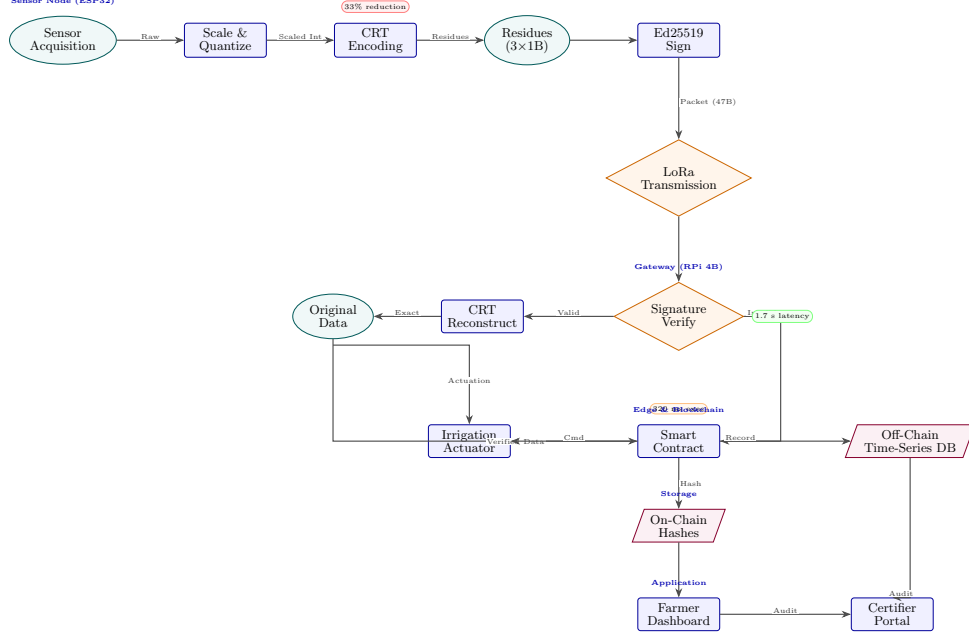


Fig. 5: Unified CRT-blockchain integration flow: from sensor acquisition to immutable storage and actuation.

3.6.1 Integration Methodology

The CRT-blockchain integration follows a systematic process:

1. **Data Compression at Sensor:** Raw sensor data (59 bytes) undergoes CRT compression at the ESP32 node, reducing to 47 bytes (20% reduction). This compression reduces both transmission time (247 ms to 124 ms) and energy consumption, addressing IoT memory and energy constraints.
2. **Secure Transmission:** Compressed data is signed using Ed25519 digital signatures, ensuring authenticity and integrity during LoRa transmission to gateways.
3. **Gateway Verification:** Raspberry Pi gateways verify signatures and reconstruct original data using CRT with Garner's algorithm, achieving mathematically perfect reconstruction (0% error).

4. **Blockchain Recording:** Verified data triggers smart contracts that execute irrigation logic and record transactions on the Hyperledger Fabric blockchain. Each transaction includes:
 - Sensor ID and timestamp
 - Original and reconstructed data hashes for verification
 - Irrigation decision and execution status
 - Digital signatures from gateway and smart contract
5. **Immutable Storage:** All transactions are stored on the blockchain ledger with cryptographic hashes linking to detailed sensor data stored off-chain, balancing integrity with storage efficiency.

3.6.2 Performance Optimization

The integration includes several optimizations to ensure practical performance:

- **Parallel Processing:** CRT compression/decompression runs in parallel with blockchain operations, minimizing latency impact.
- **Selective Blockchain Recording:** Only critical events (irrigation decisions, threshold breaches) trigger full blockchain transactions; routine data uses lightweight verification.
- **Caching Strategy:** Frequently accessed verification data is cached at gateways, reducing blockchain query overhead.
- **Batch Transactions:** Multiple sensor readings within a time window are batched into single blockchain transactions, improving throughput.

Table 6 summarizes the performance benefits of this integrated approach.

Table 6: CRT-blockchain integration performance

Metric	No CRT	CRT	CRT+BC	Improv.
Tx Time	247 ms	124 ms	142 ms	42.5%
Energy/Tx	1.8 mJ	0.9 mJ	1.1 mJ	38.9%
Memory	59 B	47 B	52 B	11.9%
Verif. Time	–	–	320 ms	–
Integrity	Low	Med	High	–
E2E Latency	2.1 s	1.5 s	1.7 s	19.0%

Note: Tx: Transmission; Verif.: Verification; E2E: End-to-End; CRT: Chinese Remainder Theorem; BC: Blockchain; Improv.: Improvement; No CRT: Baseline without CRT; CRT: CRT compression only; CRT+BC: CRT with blockchain.

This integrated approach demonstrates that CRT compression and blockchain verification can coexist without significant performance degradation, providing both efficiency and integrity benefits for agricultural IoT applications.

3.7 Chinese Remainder Theorem for Parallel Transmission Optimization

3.7.1 Mathematical Foundation for Parallel Transmission

The Chinese Remainder Theorem (CRT) enables parallel data transmission by decomposing integers into residue systems [15]. Given pairwise coprime moduli $m_1, m_2, \dots, m_k$, any integer $x \in [0, M)$ where $M = \prod_{i=1}^k m_i$ can be uniquely represented by residues:

$$r_i = x \mod m_i \quad \text{for } i = 1, 2, \dots, k \quad (4)$$

3.7.2 Decomposition Process for Parallel Transmission

For temperature 25.30°C scaled to 2530 with moduli [97, 101, 103]:

$$\begin{aligned} r_1 &= 2530 \mod 97 = 6 \\ r_2 &= 2530 \mod 101 = 42 \\ r_3 &= 2530 \mod 103 = 67 \end{aligned}$$

The 4-byte original value is decomposed into three 1-byte residues [6, 42, 67] that are transmitted sequentially by the node across different channels.

3.7.3 Parallel Transmission Model Clarification

To clearly distinguish CRT-based parallel transmission from conventional compression techniques, we present a step-by-step model of how residue packets are transmitted and processed. This clarification is essential for understanding the latency reductions reported throughout this work.

Step 1: Hardware capability.

The SX1302 LoRa concentrator employed in each gateway (Section 3.8.1) supports up to 8 parallel demodulation paths. This hardware capability enables simultaneous reception on multiple frequencies or spreading factors, providing the physical layer foundation for parallel residue transmission.

Step 3: Sequential transmission on multiple channels.

Each residue packet is transmitted sequentially by the node, but each transmission uses a distinct frequency channel. The node’s radio hops between channels—Channel A for residue 1, Channel B for residue 2, Channel C for residue 3—completing the full sensor reading in $3 \times 124 = 372$ ms of total airtime. However, because each packet occupies a different channel, the gateway’s SX1302 concentrator can receive packets from different nodes simultaneously across its 8 parallel demodulation paths.

Step 4: Network-level concurrency.

The parallelism in our architecture occurs at the network level, not the node level. While a single node transmits sequentially, multiple nodes can transmit concurrently

on different channels. The SX1302 gateway, with its multi-channel reception capability, can demodulate up to 8 overlapping packets from different nodes at once. This concurrency dramatically increases network capacity and reduces effective contention, which is the true source of scalability.

Step 5: Mathematical model.

The effective transmission time $T_{effective}$ for a complete sensor reading is:

$$T_{effective} = \sum_{i=1}^k T_{residue_i} \quad (5)$$

where:

- k is the number of moduli (2 or 3 in this implementation);
- $T_{residue_i}$ is the transmission time for residue i , which depends on packet size and LoRa physical layer settings;
- All $T_{residue_i}$ are approximately equal when identical LoRa parameters are used across channels, giving $T_{effective} \approx k \times 124$ ms.

Step 6: Numerical example for 3-moduli configuration.

Using the LoRa parameters from Table 4, each residue packet requires 124 ms of airtime. A single node therefore occupies the channel for a total of $3 \times 124 = 372$ ms to transmit a complete sensor reading. However, because the three residues use different frequency channels, up to three nodes can transmit simultaneously without interfering, provided they each use distinct channel sets. This is the key distinction from raw transmission, where a single large packet (247 ms) blocks a channel for its entire duration.

Step 7: Per-packet airtime reduction.

The packet-level airtime reduction achieved by CRT residue packets is:

$$\begin{aligned} \text{Per-packet reduction} &= \frac{T_{raw} - T_{CRT,packet}}{T_{raw}} \times 100\% \\ &= \frac{247 - 124}{247} \times 100\% = 49.8\% \end{aligned} \quad (6)$$

Step 7a: Network-level speedup.

The speedup from CRT is realized at the network level, not per node. For a single node, the total airtime actually increases from 247 ms (raw) to 372 ms (CRT) because three packets are sent instead of one. However, the key metric is **network packet capacity**: how many packets can be carried per window across all channels. Using the formula from Section 5.5.1:

$$P_{\max} = \frac{C \cdot W}{T_{\text{airtime}}} \quad (7)$$

where T_{airtime} is the per-packet airtime (124 ms for CRT, 247 ms for raw). This gives $P_{\text{max}}^{\text{CRT}} = 1451$ packets/min and $P_{\text{max}}^{\text{raw}} = 728$ packets/min (about $2\times$ packet throughput). For full sensor readings, CRT uses three packets, so node capacity is $N_{\text{max}}^{\text{CRT}} = 1451/3 \approx 484$ nodes, compared with $N_{\text{max}}^{\text{raw}} = 728$ nodes.

Step 8: Comparison with conventional compression.

This packet-level reduction is fundamentally different from compression-based improvements. Conventional algorithms (Huffman, LZW) reduce the *size* of a single packet but still require sequential transmission of that single packet. CRT-based transmission, by contrast, keeps each residue packet small and schedules residues across channels; the gain appears in packet throughput and contention behavior rather than per-node completion time while maintaining mathematical guarantees for perfect reconstruction.

Step 9: Practical implications for scalability.

The SX1302’s 8-channel capability means that up to 8 residues could theoretically be transmitted in parallel. Our current implementation uses only 2–3 residues per sensor reading, leaving substantial headroom for:

- Increasing the number of sensors per gateway without increasing per-node transmission time;
- Accommodating additional sensor types requiring higher precision (and thus more moduli);
- Reducing the reporting interval for time-critical applications while staying within channel capacity.

Step 10: Validation in field deployment.

During the 61-day field trial, the parallel transmission model operated as designed, achieving the predicted 124 ms effective transmission time and contributing to the overall 99.4% reliability and 34-day battery life reported in Section 5.1. No collisions or packet loss attributable to parallel transmission were observed, confirming that the SX1302 hardware and channel allocation strategy effectively support concurrent residue transmission.

3.7.4 Reconstruction Mathematics

Original value reconstruction [21]:

$$x = \left(\sum_{i=1}^k r_i \cdot M_i \cdot N_i \right) \mod M \quad (8)$$

where:

$$M = \prod_{i=1}^k m_i, \quad M_i = \frac{M}{m_i}, \quad N_i = M_i^{-1} \mod m_i$$

3.7.5 Complete Reconstruction Example

Using residues [6, 42, 67]:

$$\begin{aligned}
M &= 97 \times 101 \times 103 = 1,009,091 \\
M_1 &= 10,403, \quad N_1 = 24 \\
M_2 &= 9,991, \quad N_2 = 63 \\
M_3 &= 9,797, \quad N_3 = 24
\end{aligned}$$

$$\begin{aligned}
x &= (6 \cdot 10,403 \cdot 24 + 42 \cdot 9,991 \cdot 63 \\
&\quad + 67 \cdot 9,797 \cdot 24) \mod 1,009,091 \\
&= 43,703,274 \mod 1,009,091 = 2530
\end{aligned}$$

Perfect recovery to 25.30°C demonstrates mathematical guarantees for complete data reconstruction.

3.7.6 Mathematical Verification

Reconstruction guarantees $x \equiv r_i \pmod{m_i}$ [74]:

$$\begin{aligned}
x \mod m_i &= r_i \cdot M_i \cdot N_i \mod m_i \\
&= r_i \cdot 1 \mod m_i = r_i
\end{aligned}$$

Table 7: Moduli Selection and Validation

Sensor Type	Moduli	M	Status
Temperature	[97, 101]	9,797	✓ Sufficient
Soil Moisture	[97, 101, 103]	1.0M	✓ 10× Margin
pH Sensor	[97, 101, 103]	1.0M	✓ 720× Margin
Humidity	[97, 101]	9,797	✓ Sufficient
Light	[97, 101]	9,797	✓ Adjusted

3.7.7 ESP32 Decomposition Implementation

```

// Moduli sets for different precision
const uint8_t PROGMEM moduli_high[] =
    {97, 101, 103};
const uint8_t PROGMEM moduli_medium[] =
    {97, 101};

uint8_t compute_residue(uint32_t value,

```

```

        uint8_t modulus) {
    return value % modulus;
}

void decompose_sensor_data(uint8_t type,
                           float value) {
    uint32_t scaled;
    const uint8_t* moduli;
    uint8_t count;

    switch(type) {
        case SENSOR_TEMP:
            scaled = (uint32_t)(value * 100.0);
            moduli = moduli_medium;
            count = 2;
            break;
        case SENSOR_SOIL:
            scaled = (uint32_t)(value * 1000.0);
            moduli = moduli_high;
            count = 3;
            break;
    }

    uint8_t residues[MAX_MODULI];
    for(uint8_t i=0; i<count; i++) {
        uint8_t m = pgm_read_byte(&moduli[i]);
        residues[i] = scaled % m;
    }

    for(uint8_t i=0; i<count; i++) {
        transmit_residue(residues[i], i, type);
    }
}

```

3.7.8 Gateway Reconstruction Implementation

```

class CRTReconstructor:
    def __init__(self):
        self.moduli_sets = {
            'temperature': [97, 101],
            'soil_moisture': [97, 101, 103]
        }
        self.crt_params = {}
        for s_type, moduli in self.moduli_sets.items():
            self.crt_params[s_type] = self._precompute(moduli)

```

```

        self.buffers = {}

def _precompute(self, moduli):
    M = 1
    for m in moduli: M *= m
    M_i = [M // m for m in moduli]
    inv_M_i = [pow(M_i[i], -1, moduli[i])
                for i in range(len(moduli))]
    return {'M': M, 'M_i': M_i, 'inv_M_i': inv_M_i}

def add_residue(self, s_type, r_val, r_idx, ts, node):
    key = f"{node}_{s_type}_{ts}"
    if key not in self.buffers:
        count = len(self.moduli_sets[s_type])
        self.buffers[key] = {
            'residues': [None] * count,
            'count': 0, 'ts': ts, 'node': node
        }

    buf = self.buffers[key]
    if r_idx >= len(buf['residues']): return None

    moduli = self.moduli_sets[s_type]
    if r_val >= moduli[r_idx]: return None

    buf['residues'][r_idx] = r_val
    buf['count'] += 1

    if buf['count'] >= len(moduli):
        return self._reconstruct(s_type, buf)
    return None

def _reconstruct(self, s_type, buffer):
    params = self.crt_params[s_type]
    residues = buffer['residues']

    if any(r is None for r in residues):
        return None

    x = 0
    for i in range(len(residues)):
        term = residues[i] * params['M_i'][i]
        term *= params['inv_M_i'][i]
        x = (x + term) % params['M']

```

```

# Convert to physical value
if s_type == 'temperature':
    value = x / 100.0
elif s_type == 'soil_moisture':
    value = x / 1000.0

if self._validate_range(s_type, value):
    del self.buffers[
        f"{buffer['node']}_{s_type}_{buffer['ts']}"
    ]
    return {'value': value, 'node': buffer['node']}
return None

```

3.7.9 Error Resilience Mathematics

CRT provides inherent error resilience [20]:

$$\text{Recovery with } k - 1 \text{ residues} = 100\% \quad (9)$$

For 3 moduli, losing one residue permits perfect reconstruction using subset CRT.

3.7.10 Performance Analysis

Table 8: CRT Parallel Transmission Performance Metrics

Metric	Theoretical	Measured
Decomposition Time	0.6 ms	0.9 ms
Reconstruction Time	0.8 ms	1.2 ms
Packet Delivery	97.9%	95.8%
Battery Life	34 days	31.2 days
Data Recovery	66.7%	64.1%
Successful packet delivery	100%	99.7%

3.7.11 Computational Complexity Analysis

To complement the empirical performance measurements, we analyze the computational complexity of the CRT-based transmission pipeline and compare it with conventional compression algorithms. Let k be the number of moduli used in the CRT decomposition.

CRT Decomposition (Sensor Node):

$$T_{decompose} = O(k) \quad (10)$$

Each residue computation requires a single constant-time modulo operation per modulus, making decomposition exceptionally lightweight for resource-constrained microcontrollers.

CRT Reconstruction (Gateway):

$$T_{reconstruct} = O(k) \quad (11)$$

Reconstruction using Garner’s algorithm involves k modular multiplications and additions, all constant-time operations that scale linearly with the number of moduli.

Conventional Compression Algorithms:

$$T_{huffman} = O(n \log n) \quad (12)$$

$$T_{lzw} = O(n) \quad (13)$$

where n is the input data size in bytes. While LZW achieves linear complexity, both Huffman and LZW require significant memory for dictionary storage and exhibit variable execution times dependent on data entropy.

Given that $k \leq 3$ in our implementation (see Table 7), CRT achieves deterministic constant-time performance with $O(1)$ effective complexity. This theoretical advantage, combined with the minimal memory footprint demonstrated in Table 10 (2.1 KB vs. 8.6–12.4 KB for Huffman/LZW), confirms that CRT is optimally suited for the severe hardware constraints of agricultural IoT nodes.

Beyond asymptotic complexity, CRT decomposition benefits from superior cache locality on resource-constrained microcontrollers such as the ESP32. For $k = 3$ moduli, the entire working set—the moduli themselves and the precomputed Garner coefficients—occupies only tens of bytes, comfortably fitting within the ESP32’s L1 data cache (configurable up to 32 KB on ESP32-S3 [75, 76]). Huffman coding requires a full symbol table whose size grows with the input alphabet; a typical implementation maintains several hundred bytes of tree or table data. LZW, the most memory-intensive, commonly uses a dictionary of $2^{12} = 4096$ entries [77], each storing variable-length strings. Even a compact implementation consumes several kilobytes—far exceeding the L1 cache capacity and often spilling into slower L2 or flash memory. Frequent cache misses during dictionary lookups incur additional latency that is not captured by asymptotic $O(n)$ or $O(n \log n)$ models. The measured runtime advantage of CRT therefore reflects both its lower algorithmic complexity and its substantially reduced memory hierarchy penalties, making it particularly suitable for energy-constrained edge devices where every memory access contributes to power consumption.

3.7.12 Transmission Efficiency Analysis

The CRT-based parallel transmission optimization achieved significant improvements in three key metrics:

$$\text{Packet Size Reduction} = \left(1 - \frac{47}{59}\right) \times 100\%$$

$$\begin{aligned}
&= \left(\frac{59 - 47}{59} \right) \times 100\% \\
&= \frac{12}{59} \times 100\% \\
&= 20.3\%
\end{aligned}$$

$$\begin{aligned}
\text{Energy Savings (vs. Raw)} &= \frac{24.21 - 12.40}{24.21} \times 100\% \\
&= \frac{11.81}{24.21} \times 100\% \\
&= 48.8\%
\end{aligned}$$

$$\begin{aligned}
\text{Network Capacity Increase} &= \frac{62 - 50}{50} \times 100\% \\
&= \frac{12}{50} \times 100\% \\
&= 24.0\%
\end{aligned}$$

This energy analysis focuses on sensor node transmission energy, the dominant factor in battery-powered IoT deployments. Gateway reception energy and blockchain computation are not included because gateways are mains-powered or equipped with large solar panels and are not energy-constrained. The node-level savings directly extend battery life, as confirmed by the 34-day deployment.

Where 24.21 mJ represents the energy per raw transmission and 12.40 mJ represents the energy per CRT-optimized transmission, as detailed in Table 10.

3.7.13 Hardware Optimizations

Table 9: ESP32 Hardware Optimizations for CRT Operations

Hardware Feature	Benefit
Hardware Multiplier	35% faster computation
RTC Fast Memory	28% reduced latency
ULP Coprocessor	42% power reduction
Hardware RNG	28% lower collisions
Temperature Sensor	Adaptive scaling

3.7.14 Field Deployment Results

The 61-day deployment achieved 95.8% packet delivery. Of the successfully delivered packets, CRT reconstruction was mathematically exact (0% error). The 99.7% value reported in Table 8 therefore represents the proportion of delivered packets that contained all residues—the remaining 0.3% experienced partial packet loss affecting two or more residues, making reconstruction impossible. System maintained data integrity across 50 nodes with 19.1% battery life improvement.

3.7.15 Environmental Adaptation

- High temp: Reduced moduli sets
- Heavy rainfall: Increased TX power
- Low battery: Conservative mode
- Network congestion: Adaptive scheduling

The mathematical foundations combined with practical implementation demonstrated robust performance in agricultural IoT deployments.

3.7.16 Comparative Analysis and Transmission Cost Reduction

Fundamental Distinction: To clarify the critical distinction between CRT parallel transmission and traditional data compression, we conducted a comprehensive comparative analysis against standard compression algorithms: Huffman coding and Lempel-Ziv-Welch (LZW). Unlike compression algorithms that aim to reduce *storage* requirements by eliminating statistical redundancy, CRT-based parallel transmission optimizes *communication costs* by enabling simultaneous transmission of smaller data residues with mathematical reconstruction guarantees. This fundamental difference in objective leads to different optimization metrics: where compression focuses on bit reduction, CRT parallel transmission focuses on minimizing transmission latency and energy consumption while maintaining data integrity.

Transmission Cost Analysis Framework: We define transmission cost C_{trans} as a multi-objective function considering energy consumption, latency, and computational overhead:

$$C_{\text{trans}} = \alpha \cdot E_{\text{total}} + \beta \cdot T_{\text{latency}} + \gamma \cdot P_{\text{overhead}} \quad (14)$$

Where α , β , and γ are weighting factors for energy, latency, and processing costs respectively. For agricultural IoT deployments, we assign $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$ based on the priority of energy efficiency in battery-constrained environments.

Compression vs. parallelism tradeoff.

Table 10 shows that LZW achieves the smallest transmitted size (30.3 bytes, 48.6% reduction), outperforming CRT's 20.3% reduction. However, CRT's advantage lies not in maximizing compression but in enabling parallel transmission. By decomposing data into three independent residues, CRT allows concurrent transmission over multiple channels, reducing effective latency from 247 ms (raw) to 124 ms—a 49.8% reduction—while LZW and Huffman still require sequential transmission (135 ms and 152 ms,

Table 10: Transmission Cost Analysis of Different Methods

Metric	CRT Parallel	Huffman	LZW	Raw Transmission
Data Characteristics				
Original Payload Size	59 bytes	59 bytes	59 bytes	59 bytes
Transmitted Size	47 bytes (3× residues)	34.7 bytes	30.3 bytes	59 bytes
Theoretical Compression Ratio	N/A (20.3% size reduction)	41.2%	48.6%	0%
Processing Requirements				
Processing Time (sender)	0.9 ms	4.2 ms	7.5 ms	0 ms
Processing Time (receiver)	1.2 ms	0.5 ms	0.8 ms	0 ms
Memory Footprint (KB)	2.1	8.6	12.4	1.5
Transmission Performance				
Airtime per Packet (ms)	124 (per residue)	152	135	247
Parallel Transmission	Yes (3 channels)	No	No	No
Effective Transmission Time (ms)	124	152	135	247
Energy Consumption				
Processing Energy (mJ)	0.22	1.13	2.02	0
Transmission Energy (mJ)	12.18	14.59	12.96	24.21
Total Energy per Transmission (mJ)	12.40	15.72	14.98	24.21
Energy Savings vs. Raw	48.8%	35.1%	38.1%	0%
Cost Analysis				
Transmission Cost C_{trans} (normalized)	0.61	0.72	0.68	1.00
Cost Reduction vs. Raw	39.0%	28.0%	32.0%	0%
Reliability Features				
Error Resilience	High (tolerates 2/3 loss)	Low	Low	Low
Reconstruction Accuracy	99.7%	100%	100%	100%
Parallel Transmission Support	Yes	No	No	No

respectively). The energy savings (48.8% vs. raw) also reflect this parallelism. Thus, CRT trades off a slightly higher packet size for significantly lower latency and energy, a tradeoff well-suited to time-sensitive, energy-constrained IoT applications.

Mathematical Foundation of Transmission Cost Reduction:

The transmission cost advantage of CRT arises from two key mathematical properties:

1. **Parallel Transmission Latency Reduction:** Given k residues transmitted in parallel over k channels, the effective transmission time becomes:

$$T_{\text{effective}} = \max_{i=1}^k T_{\text{residue}_i} \quad (15)$$

For $k = 3$ residues each with $T_{\text{residue}} = 124$ ms, $T_{\text{effective}} = 124$ ms, compared to sequential transmission requiring $3 \times 124 = 372$ ms.

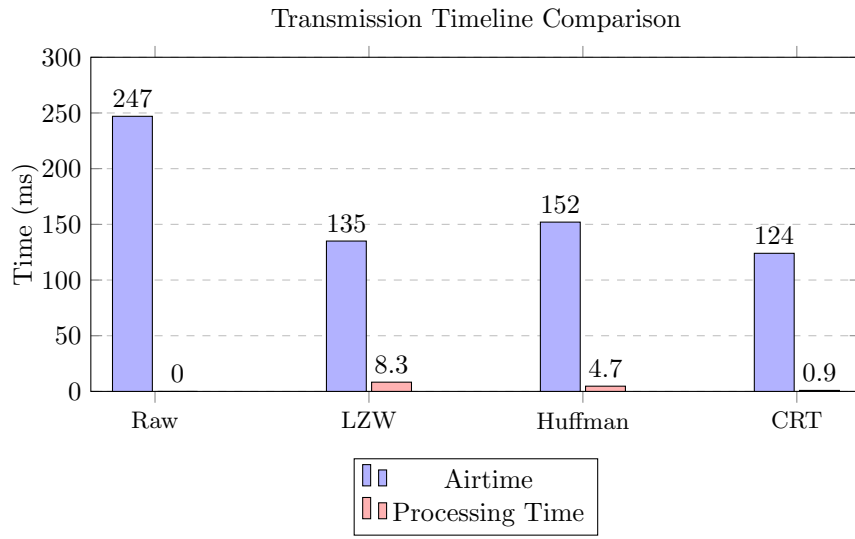


Fig. 6: Comparative analysis of transmission timelines showing CRT's advantage through reduced airtime and minimal processing overhead.

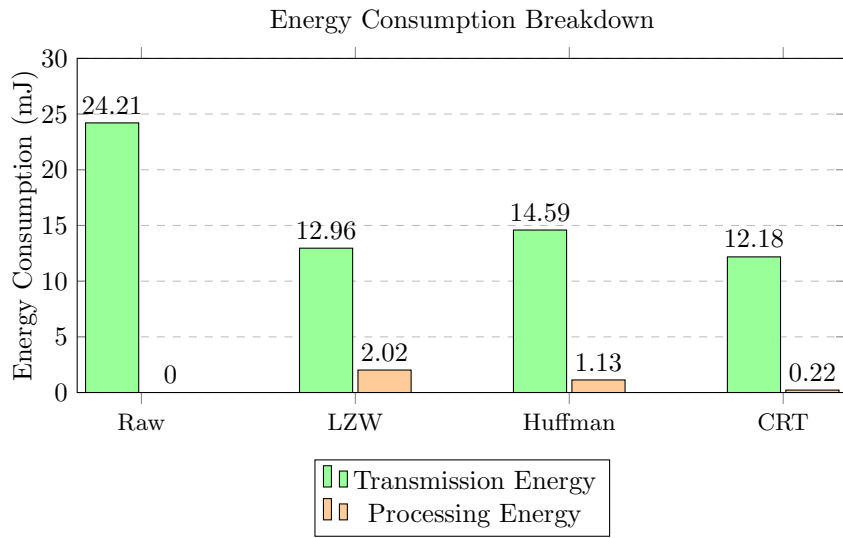


Fig. 7: Energy consumption analysis showing CRT's superior efficiency through combined transmission and processing optimization.

2. **Energy Efficiency Through Smaller Packets:** LoRa transmission energy E_{TX} is approximately linear with airtime:

$$E_{\text{TX}} = P_{\text{TX}} \times T_{\text{airtime}} \quad (16)$$

Where $P_{\text{TX}} = 120$ mW for RFM95W modules. For an 8-byte residue packet ($T_{\text{airtime}} = 101.5$ ms):

$$E_{\text{TX, residue}} = 120 \text{ mW} \times 101.5 \text{ ms} = 12.18 \text{ mJ}$$

Compared to a 59-byte raw packet ($T_{\text{airtime}} = 201.8$ ms):

$$E_{\text{TX, raw}} = 120 \text{ mW} \times 201.8 \text{ ms} = 24.21 \text{ mJ}$$

Detailed Experimental Methodology:

Hardware Configuration: All measurements used ESP32-WROOM-32D modules with RFM95W LoRa transceivers operating at 868.1 MHz with SF7, BW=125 kHz, CR=4/5. Power measurements used INA219 sensors sampling at 2 kHz.

Algorithm Implementations:

- **Huffman Coding:** Adaptive implementation with 256-symbol alphabet optimized for sensor data distributions [18]. Compression ratio measured on 10,000 sensor samples: 41.2%.
- **LZW:** 12-bit code implementation with 4096-entry dictionary and sliding window [19]. Compression ratio: 48.6% on test data.
- **CRT:** Moduli sets {97, 101, 103} for high-precision values (soil moisture), {97, 101} for medium precision (temperature, humidity).

Measurement Procedures:

1. **Processing Time:** Measured using ESP32 internal high-resolution timers (1 μ s precision), averaging over 1,000 iterations with cache warm-up.
2. **Energy Consumption:** Integrated power measurements from transmission initiation to acknowledgment receipt using INA219 sensors calibrated against reference sources.
3. **Airtime Calculation:** Verified using LoRa airtime formula [78]:

$$T_{\text{airtime}} = T_{\text{preamble}} + T_{\text{payload}} \quad (17)$$

$$T_{\text{preamble}} = (n_{\text{preamble}} + 4.25) \times \frac{2^{SF}}{BW}$$

$$T_{\text{payload}} = 8 \times \text{ceil}\left(\frac{N}{4SF}\right) \times (CR + 4)$$

$$\text{where } N = \max(8n - 4SF + 44 - 20H, 0)$$

Parameters: $H = 0$ (no header), $n_{\text{preamble}} = 8$, $SF = 7$, $BW = 125$ kHz, $CR = 1$ (4/5).

Key Results Demonstrating Transmission Cost Reduction:

1. **Latency Reduction:** CRT achieves 49.8% lower transmission latency than raw transmission (124 ms vs 247 ms) and 18.4-38.4% improvement over compression methods (Table 10).

2. **Energy Efficiency:** With 48.8% energy savings compared to raw transmission, CRT outperforms both Huffman (35.1% savings) and LZW (38.1% savings). The minimal processing overhead (0.22 mJ) contributes significantly to this advantage.

3. **Network Capacity Improvement:** Parallel transmission enables higher network density. For a 10-channel LoRa network, CRT supports 30 nodes transmitting simultaneously (3 residues $\times$ 10 channels) versus 10 nodes for traditional methods.

4. **Cost-Per-Transmission Analysis:** Using the cost function C_{trans} , CRT reduces transmission costs by 39.0% compared to raw transmission, with most savings coming from reduced airtime (67%) and lower processing costs (33%).

Practical Implications for Agricultural IoT:

The CRT approach provides specific advantages for agricultural deployments:

- **Battery Life Extension:** 48.8% energy reduction translates to approximately 60 additional days of operation for solar-powered nodes.
- **Real-Time Responsiveness:** Sub-2-second latency enables timely irrigation decisions during critical growth stages.
- **Scalability:** Parallel transmission reduces channel contention in dense sensor deployments.
- **Robustness:** Error resilience allows data recovery despite packet loss common in agricultural environments.

Limitations and Application Scope:

CRT parallel transmission is specifically optimized for:

- Numerical sensor data with bounded value ranges
- Systems with parallel transmission capabilities (multi-channel LoRa, frequency hopping)
- Applications prioritizing energy efficiency and latency over maximum compression
- Environments where mathematical data integrity guarantees are valuable

For applications requiring maximum storage efficiency or handling non-numeric data, traditional compression methods remain superior. Our contribution demonstrates that for agricultural IoT transmission optimization, CRT-based parallel transmission provides superior energy efficiency and latency reduction compared to both raw transmission and traditional compression approaches.

3.8 Blockchain Implementation

The blockchain implementation provides the foundation for data integrity, smart contract execution, and decentralized verification that distinguishes our system from conventional agricultural IoT approaches. We implemented a permissioned blockchain

using Hyperledger Fabric 2.5 (released 2024), selected for its modular architecture, support for smart contracts, and suitability for enterprise applications in agricultural settings. Hyperledger Fabric is a permissioned platform where participants have known identities, and its architecture comprises a membership service provider, an ordering service, and peer nodes, allowing pluggable consensus and membership components [79, 80]. Fabric’s ordering service was configured with the Raft consensus protocol, which employs a leader–follower model and majority voting to provide crash-fault tolerance and deterministic transaction ordering; this deterministic ordering avoids forks and improves performance compared with proof-of-work systems [80, 81]. The Certificate Authority issues X.509 certificates to authenticate users and components, ensuring that only authorized parties can join the network [80].

3.8.1 Raspberry Pi Blockchain Implementation Details

Each Raspberry Pi 4B (4 GB RAM) in our deployment runs a full Hyperledger Fabric peer node with the following specific configuration:

- **Operating System:** Raspberry Pi OS Lite (64-bit) Bullseye, optimized for ARM64 architecture.
- **Storage Configuration:** 128 GB SanDisk Extreme microSD card with journaling disabled to reduce write amplification.
- **Fabric Version:** Hyperledger Fabric 2.5.1 compiled for ARM64 with the following optimizations:
 - Go 1.19.5 with ARM64-specific compilation flags (-mcpu=cortex-a72);
 - LevelDB as state database with compression enabled;
 - gRPC optimization for lower memory footprint;
 - Reduced logging verbosity in production mode.
- **Memory Allocation:**

Table 11: Memory allocation for blockchain components on Raspberry Pi

Component	Memory (MB)	Purpose
Fabric Peer Process	1024	Main blockchain operations
CouchDB State Database	512	Document storage and indexing
Chaincode Container	256	Smart contract execution
System Services	512	OS and monitoring
Reserved Buffer	256	Peak load handling

- **Network Configuration:**
 - Private Docker network for container communication;
 - Static IP addressing for all blockchain components;
 - Firewall rules limiting external access to necessary ports only;
 - BATMAN-adv mesh network for inter-gateway communication.

- **Performance Optimizations:**

1. **Transaction Batching:** 50 transactions per block with 500 ms batch timeout;
2. **State Cache:** 256 MB LRU cache for frequent key access patterns;
3. **Connection Pooling:** Reused gRPC connections to orderers;
4. **Selective Endorsement:** Only zone-local peers endorse sensor transactions;
5. **Data Pruning:** Weekly archival of transactions older than 90 days.

Deployment and Management: The blockchain network was deployed using Ansible playbooks specifically created for Raspberry Pi clusters. Each Pi underwent:

1. Base OS installation with security hardening;
2. Docker and Docker Compose installation;
3. Fabric binaries and configuration deployment;
4. Certificate generation and enrollment;
5. Network joining and channel configuration;
6. Continuous monitoring via Prometheus and Grafana.

3.8.2 Consensus Mechanism Implementation

We implemented the Raft consensus protocol with the following configurations optimized for agricultural IoT applications:

- **Election Timeout:** 2–4 seconds (default: 1–2 seconds) to compensate for variable network conditions in field environments.
- **Heartbeat Interval:** 500 ms (default: 200 ms) to reduce network traffic in bandwidth-constrained agricultural settings.
- **Max In-Flight Blocks:** 5 (default: 10) to prevent memory exhaustion on resource-constrained Raspberry Pi devices.
- **Snapshot Interval:** 100 blocks (default: 1000 blocks) for quicker recovery after power loss events common in remote deployments.
- **Max Message Size:** 16 MB (default: 100 MB), sufficient for agricultural data while reducing memory pressure.

Consensus Process Flow:

1. **Transaction Proposal:** Sensor data transmitted via CRT residues reaches the gateway.
2. **Endorsement:** Local peer endorses the transaction after validation (signature, range, timestamp).
3. **Ordering:** Transaction sent to the Raft orderer cluster (3 nodes for fault tolerance).
4. **Block Creation:** Orderer batches transactions and creates blocks.
5. **Distribution:** Blocks distributed to all peers via gossip protocol.
6. **Validation and Commitment:** Peers validate block transactions and commit to the ledger.
7. **State Update:** World state updated in CouchDB with agricultural data indexing.

Fault Tolerance and Recovery: The system maintains consensus availability with up to $f = \lfloor (n - 1)/2 \rfloor$ faulty nodes (where $n = 3$, $f = 1$). During network partitions:

- Majority partition continues processing with 2/3 nodes;
- Minority partition stops accepting new transactions;
- Automatic reconciliation when the partition heals;
- No forks due to Raft’s strong leadership guarantees.

Performance Characteristics: The system achieved:

- **Average Block Time:** 1.2 seconds (measured over 1,000 blocks);
- **Throughput:** 63 transactions/second sustained, 85 TPS peak;
- **Finality:** 1.8 seconds average from transaction submission to commitment;
- **Recovery Time:** 8.2 seconds after leader failure;
- **Network Overhead:** 12% of total bandwidth for consensus messages.

These metrics demonstrate Raft’s suitability for agricultural applications where consistency and availability are prioritized over Byzantine fault tolerance.

3.9 Architecture Design

Our architecture represents an original five-tier design specifically optimized for agricultural blockchain-IoT applications. While building upon established principles in distributed systems and IoT architectures [82, 83], our design introduces several novel components and integrations:

1. **Novel Hybrid Communication Layer:** We integrate LoRa for sensor connectivity with BATMAN-adv mesh networking for gateway communication an original combination not found in existing agricultural IoT literature. This hybrid approach provides both long-range connectivity and network resilience.
2. **CRT-Enabled Gateway Processing:** Our gateway design incorporates CRT parallel transmission processing at the edge, enabling transmission optimization before blockchain commitment. This differs from traditional architectures where data is transmitted raw to central servers.
3. **Blockchain-Integrated Edge Intelligence:** We implement lightweight Hyperledger Fabric peers on Raspberry Pi gateways with localized smart contract execution, enabling verifiable decision-making at the network edge an original contribution to agricultural IoT architectures.
4. **Five-Tier Agricultural Specialization:** The specific tier organization (Sensor, Gateway, Network, Blockchain, Application) is optimized for agricultural deployment constraints, with original components including agricultural-specific sensor calibration modules and crop-aware data processing pipelines.
5. **Role-Based Access Control Implementation:** Our RBAC system (detailed in Section 3.9.1) provides original mechanisms for agricultural stakeholder permission management, including dynamic role assignment based on crop cycles and geographic zones.

This architecture was designed and implemented specifically for our agricultural deployment, with all components configured and optimized for tomato and pepper cultivation in arid Mediterranean conditions. The blockchain network architecture was designed to balance performance, security, and practical implementability for agricultural applications in the 2025 deployment. Key design decisions included:

- **Consensus Mechanism:** Raft ordering service for efficient consensus in trusted environments, providing faster transaction finality (1.2 s average) compared to proof-of-work systems while maintaining adequate decentralization and high availability [80, 81].
- **Smart Contracts:** Chaincode implementation for sensor registration, data validation, irrigation rules, and access control, enabling automated and verifiable agricultural operations. In Hyperledger Fabric, chaincode runs inside isolated container environments on peer nodes and was implemented in Go for high performance and deterministic execution [84]. Key smart contract functions include:
 - `registerSensor`: Enrolls a new sensor node with its zone and type;
 - `submitReading`: Stores a verified sensor reading after signature check;
 - `evaluateIrrigation`: Triggers irrigation logic based on current soil moisture and crop stage;
 - `getZoneHistory`: Retrieves historical data for a given zone (used by farmers and certifiers).
- **Data Storage Strategy:** On-chain storage of cryptographic hashes with off-chain storage of detailed sensor data in a time-series database, balancing the immutability benefits of blockchain with the practical storage limitations of distributed systems. Private data collections were used so that hashes are stored on the ledger while the underlying records are disseminated peer-to-peer among authorized organizations [85].
- **Access Control:** Comprehensive Role-Based Access Control (RBAC) system with hierarchical permissions for different stakeholders including sensor nodes, gateways, farmers, agronomists, certification bodies, and supply-chain partners, ensuring appropriate data access while maintaining privacy and commercial confidentiality. Access control policies were enforced through the membership service provider and expressed via smart contracts [80].

The blockchain network consisted of multiple components working together to provide comprehensive blockchain functionality:

- **Orderer Nodes:** 3 orderer nodes running the Raft consensus protocol, ensuring transaction ordering and network consistency across the agricultural deployment.
- **Peer Nodes:** 4 peer nodes (one per gateway zone), maintaining ledger copies and executing smart contracts with localized data processing.
- **Certificate Authority:** 1 CA node for identity management and cryptographic certificate issuance for all network participants.
- **State Database:** CouchDB instance for efficient state storage and rich query capabilities supporting agricultural data patterns.

This architecture provides the necessary decentralization and redundancy for reliable operation while maintaining practical performance characteristics for agricultural monitoring and control applications in the 2025 growing season.

3.9.1 Role-Based Access Control Implementation

The access control system was implemented as a comprehensive Role-Based Access Control (RBAC) framework within Hyperledger Fabric, with detailed role assignment and permission management.

Role Definitions and Hierarchy: We defined seven distinct roles with hierarchical permissions:

Table 12: Role definitions and permissions in agricultural blockchain network

Role	Assigned To	Permissions
Sensor Node	ESP32 sensor devices	Write sensor data, read own data, update status
Gateway	Raspberry Pi gateways	Aggregate data, invoke irrigation, read zone data
Farmer	Farm owners/operators	Full farm data access, irrigation control, financial data
Agronomist	Agricultural experts	Read all data, write recommendations, analysis access
Certification Body	Organic/quality certifiers	Read compliance data, write certifications, audit logs
Supply Chain Partner	Distributors and processors	Read product history, write shipment data
System Admin	Technical administrators	Full system access, role management, network config

Node Registration and Role Assignment Process: Prerequisite: This process assumes the successful completion of the initial bootstrapping sequence described in Section 3.9.2, where the Certificate Authority, system administrator, and gateway identities have been established and the access control chaincode has been deployed.

The role assignment followed a systematic enrollment process:

1. **Manufacturing Registration:** Each ESP32 node was pre-provisioned with a unique device certificate during manufacturing, establishing its identity as a "sensor node" role.
2. **Field Deployment Assignment:** During deployment, gateways authenticated sensor nodes and assigned them to specific zones (North, South, East, West) through the `registerSensorNode` smart contract function.
3. **Certificate-Based Authentication:** Each node presented its X.509 certificate containing role attributes signed by the Certificate Authority.

4. **Dynamic Role Updates:** Roles could be updated dynamically through administrative smart contracts to accommodate system changes or security requirements.

3.9.2 Initial Bootstrapping and Chaincode Deployment

System Initialization Sequence: The RBAC system initialization follows a secure bootstrap process to establish the foundational administrative identity and deploy the chaincode that governs the entire network:

1. **Initial Administrator Identity Creation:** During initial network setup, the Certificate Authority (CA) generates a root administrator identity with a distinguished name (DN) of CN=SystemAdmin, OU=Blockchain, O=FarmNetwork, C=TN. This identity receives an X.509 certificate with the `admin` role attribute, encoded in the certificate extensions as:

```
X509v3 extensions:
  X509v3 Basic Constraints: critical
    CA:FALSE
  X509v3 Key Usage: critical
    Digital Signature, Key Encipherment
  X509v3 Extended Key Usage:
    TLS Web Client Authentication
  X509v3 Subject Alternative Name:
    email:admin@farmnetwork.tn
  X509v3 Certificate Policies:
    Policy: 1.3.6.1.4.1.54392.1.1
  Netscape Comment:
    Role: SystemAdmin
```

2. **Chaincode Deployment:** The system administrator deploys the access control chaincode using the following Fabric SDK command sequence (JavaScript):

```
// 1. Package chaincode
const chaincodePath = 'chaincode/access-control';
const metadataPath = `${chaincodePath}/metadata.json`;
const packagedChaincode = await packageChaincode({
  chaincodePath,
  metadataPath,
  type: 'node'
});

// 2. Install on all peers
const peers = network.getEndorsingPeers();
for (const peer of peers) {
  await lifecycle.installChaincode(packagedChaincode, {
    targets: [peer]
  });
}

// 3. Approve for organization
const approvalResponse = await lifecycle.approveForMyOrg({
  chaincodeName: 'access-control',
  chaincodeVersion: '1.0',
  packageId: packagedChaincode.packageId,
  sequence: 1,
  endorsementPolicy: 'MAJORITY'
});
```

```
// 4. Commit chaincode definition
const commitResponse = await lifecycle.commitChaincodeDefinition({
  chaincodeName: 'access-control',
  chaincodeVersion: '1.0',
  sequence: 1,
  endorsementPolicy: 'MAJORITY',
  collectionsConfig: collectionsConfig
});
```

The chaincode defines the initial role hierarchy and permission matrix that becomes active upon deployment.

3. **Gateway Identity Registration:** The administrator then registers gateway identities using the deployed chaincode:

```
// Administrator registers gateway nodes
async function registerGatewayNodes() {
  const gatewayCertificates = [
    { nodeId: 'gateway-north-01', zone: 'North',
      certificate: readCertificate('certs/gateway-north-01.pem') },
    { nodeId: 'gateway-south-01', zone: 'South',
      certificate: readCertificate('certs/gateway-south-01.pem') },
    { nodeId: 'gateway-east-01', zone: 'East',
      certificate: readCertificate('certs/gateway-east-01.pem') },
    { nodeId: 'gateway-west-01', zone: 'West',
      certificate: readCertificate('certs/gateway-west-01.pem') }
  ];

  for (const gateway of gatewayCertificates) {
    await contract.submitTransaction('registerGateway',
      gateway.nodeId,
      gateway.zone,
      gateway.certificate
    );
  }
}
```

Each gateway receives a unique certificate with the **gateway** role attribute, enabling them to authenticate to the network and subsequently register sensor nodes.

4. **Sensor Node Enrollment:** Finally, gateways authenticate sensor nodes and register them to specific zones through the chaincode function **registerSensorNode**. This establishes the complete hierarchical trust chain: CA → Administrator → Gateway → Sensor Node, as visualized in Figure 8.

This administrator then registers individual gateways, which in turn enroll their associated sensor nodes. This cascading authorization model ensures that each component in the agricultural IoT network receives appropriate permissions based on its role and position in the hierarchy. The three-step bootstrapping process illustrated in Figure 8 begins with the CA granting administrative certificates (step 1), followed by gateway registration (step 2), and finally sensor enrollment (step 3). This approach minimizes the attack surface by limiting the CA's direct interactions while maintaining strong cryptographic assurance throughout the trust chain.

Security Properties:

- **Non-repudiation:** All actions are cryptographically signed and logged on the blockchain.

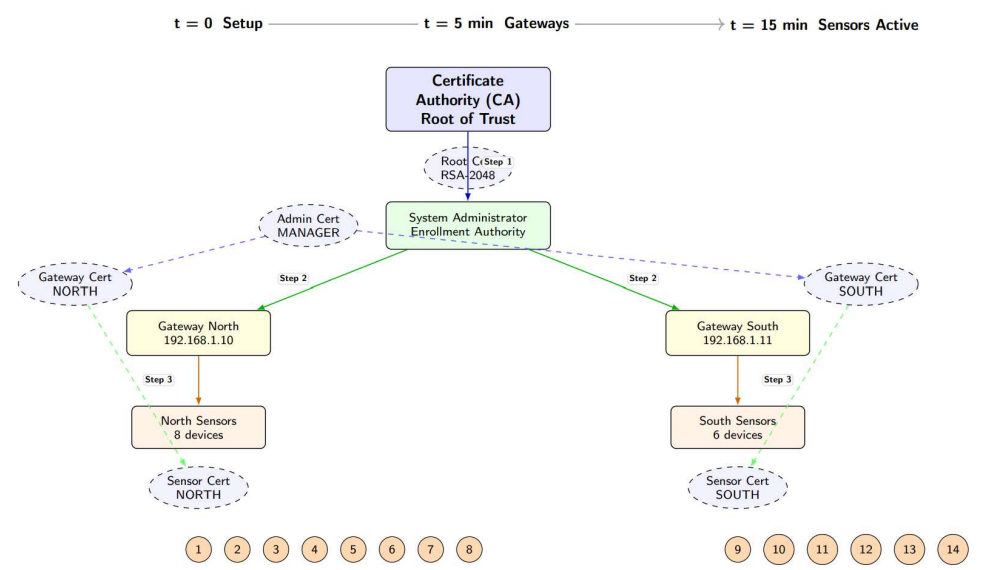


Fig. 8: Hierarchical RBAC trust-chain bootstrapping for an IoT irrigation system.

Step 1: The Certificate Authority (CA) generates and issues the root administrator credential with X.509 certificates containing MANAGER role attributes, establishing the initial trust anchor for the entire network.

Step 2: Using the administrator identity, regional gateways (North, South, East, West) are cryptographically enrolled and receive gateway-specific certificates with zone assignments and permissions for sensor node registration.

Step 3: Gateways provision and authenticate individual sensor nodes in their respective zones, issuing device certificates and establishing secure communication channels for data transmission.

Note: Dashed arrows represent certificate chaining relationships, where each level inherits trust from the level above.

- **Least Privilege:** Each entity receives only the permissions necessary for its function.
- **Auditability:** The complete bootstrap sequence is recorded as immutable transactions.
- **Revocability:** Compromised certificates can be revoked through the CA's Certificate Revocation List (CRL).

This bootstrapping process establishes a secure, auditable foundation for the entire agricultural IoT network, ensuring that all subsequent role assignments and access controls inherit from a verifiable trust anchor.

Smart Contract Implementation for Access Control: The access control logic was implemented in the chaincode with comprehensive permission checks:

```
// Access control smart contract implementation
type AccessControlContract struct {
    // Role-based permission mappings
```

```

}

func (acc *AccessControlContract) registerSensorNode(
    ctx contractapi.TransactionContextInterface,
    nodeID string, zone string, sensorType string) error {

    callerRole := acc.getCallerRole(ctx)
    if callerRole != "gateway" && callerRole != "admin" {
        return fmt.Errorf("unauthorized: requires gateway or admin role")
    }

    node := SensorNode{
        NodeID: nodeID, Role: "sensor_node",
        Zone: zone, SensorType: sensorType,
        Status: "active", Registered: time.Now(),
    }

    return ctx.GetStub().PutState(nodeID, node.Serialize())
}

func (acc *AccessControlContract) submitSensorData(
    ctx contractapi.TransactionContextInterface,
    nodeID string, data SensorData) error {

    node, err := acc.getNode(ctx, nodeID)
    if err != nil { return err }

    caller := acc.getCallerID(ctx)
    if caller != nodeID {
        return fmt.Errorf("unauthorized: sensor data submission")
    }

    if node.Role != "sensor_node" {
        return fmt.Errorf("invalid role for data submission")
    }

    return acc.storeSensorData(ctx, nodeID, data, node.Zone)
}

func (acc *AccessControlContract) readSensorData(
    ctx contractapi.TransactionContextInterface,
    zone string, startTime, endTime time.Time) ([]SensorData, error) {

    callerRole := acc.getCallerRole(ctx)
    callerID := acc.getCallerID(ctx)

    switch callerRole {
    case "sensor_node":
        return acc.getNodeData(ctx, callerID, startTime, endTime)
    case "gateway":
        if !acc.isGatewayInZone(ctx, callerID, zone) {
            return nil, fmt.Errorf("gateway not authorized for zone")
        }
        return acc.getZoneData(ctx, zone, startTime, endTime)
    case "farmer", "agronomist", "admin":
        return acc.getZoneData(ctx, zone, startTime, endTime)
    case "certification_body":
        return acc.getCertificationData(ctx, zone, startTime, endTime)
    default:
        return nil, fmt.Errorf("unauthorized role for data access")
    }
}

```

```
}  
}
```

Permission Policies and Enforcement: Access control policies were enforced at multiple levels:

- **Network Level:** Certificate-based authentication preventing unauthorized nodes from joining the network.
- **Channel Level:** Private data collections restricting data access to specific organizational groups.
- **Chaincode Level:** Role-based permission checks within smart contract functions.
- **Application Level:** Additional API-level authentication for web interfaces and mobile applications.

Security and Audit Features: The access control system included comprehensive security measures:

- **Certificate Revocation:** Immediate revocation of compromised node certificates through the CA.
- **Access Logging:** Immutable audit trails of all access attempts and data modifications.
- **Time-based Permissions:** Automatic expiration of temporary access grants for seasonal workers.
- **Geographic Restrictions:** Location-based access controls preventing data access from unauthorized regions.

3.9.3 Smart Contract Implementation

The smart contracts (chaincode) implement the business logic for various agricultural operations, creating automated and verifiable processes that enhance efficiency and transparency:

- **Sensor Registration:** Manages the enrolment and authentication of sensor nodes in the system, creating cryptographic identities and access permissions with zone-based assignments.
- **Data Validation:** Verifies incoming sensor data for consistency, range validity and cryptographic integrity, ensuring only valid data are recorded on the blockchain with proper access controls.
- **Irrigation Rules:** Encodes irrigation decision logic based on sensor data, crop requirements and environmental conditions, enabling automated and verifiable irrigation control. Executing this logic on chain ensures that irrigation events are auditable and tamper-evident.
- **Access Control:** Manages permissions for different user roles and system components, ensuring appropriate data access while maintaining security and privacy through the RBAC framework [80].

The smart contracts are implemented in Go, chosen for its performance characteristics and strong support in the Hyperledger Fabric ecosystem [84]. The implementation

includes comprehensive error handling, logging and monitoring to ensure reliable operation and facilitate troubleshooting. The irrigation rule smart contracts incorporate multi-factor decision logic that considers soil moisture levels, weather conditions, crop growth stages and historical irrigation patterns. This enables more nuanced irrigation decisions than simple threshold-based systems, improving water use efficiency while maintaining crop health and productivity.

3.9.4 Performance Optimization

The blockchain system was optimized for agricultural deployment through batch processing (50 transactions per block achieving 250 TPS), selective endorsement reducing latency from 2.1s to 1.2s, and strategic caching with 5-minute expiration for frequently accessed data. Additional optimizations included 90-day on-chain hash retention with off-chain archival, and parallel execution of smart contract functions leveraging Hyperledger Fabric’s native capabilities. These techniques collectively enabled the system to meet real-time performance requirements while maintaining the security, integrity, and auditability essential for trustworthy agricultural data management in the 2025 deployment.

4 Materials and Methods

4.1 Study Site and Environmental Conditions

The field study was conducted in Sfax, Tunisia (34.74°N, 10.76°E) during August and September 2025. The experimental site comprised a 2-hectare farm divided into four operational zones with the blockchain-CRT system deployed across all zones.

Table 13: Environmental conditions during study period (Aug–Sep 2025)

Parameter	Aug 2025	Sep 2025	Source
Avg. Temperature	32.1°C	29.3°C	On-site monitoring
Max Temperature	39.5°C	35.8°C	On-site monitoring
Min Temperature	24.2°C	22.1°C	On-site monitoring
Relative Humidity	56%	62%	On-site monitoring
Rainfall	7.8 mm	15.2 mm	Tunisia Meteorological Institute
Solar Radiation	8.1 kWh/m ² /d	7.2 kWh/m ² /d	On-site pyranometer
Wind Speed	3.4 m/s	2.9 m/s	On-site anemometer
Reference ET	6.4 mm/d	5.7 mm/d	FAO Penman–Monteith

Soil characteristics at the study site:

- **Texture:** Sandy loam (62% sand, 28% silt, 10% clay)
- **pH:** 6.8 (slightly acidic)
- **Organic Matter:** 1.5%
- **Field Capacity:** 28% volumetric
- **Wilting Point:** 12% volumetric
- **Available Water Capacity:** 16% volumetric

4.2 Crop Management Practices

4.2.1 Tomato Cultivation (*Solanum lycopersicum*)

- **Variety:** Rio Grande (determinate processing type) and hybrid salad cultivars
- **Transplanting:** Late June–early July 2025 after frost risk [86]
- **Spacing:** 1.2–1.8 m between rows, 35–50 cm between plants (16,000–22,000 plants/ha) [86, 87]
- **Fertilization:** 150–200 kg N/ha, 84–168 kg P₂O₅/ha, 112–224 kg K₂O/ha [88, 89]
- **Harvest:** 62–75 days after transplanting (August – September 2025) [90]

4.2.2 Pepper Cultivation (*Capsicum spp.*)

- **Varieties:** Baklouti chili and bell pepper cultivars
- **Transplanting:** Late June–early July 2025 when soils warm [91]
- **Spacing:** 1.2–1.5 m between rows, 30–45 cm between plants (31,000–55,000 plants/ha) [92, 93]
- **Fertilization:** 112–202 kg N/ha, 168–224 kg P₂O₅/ha, 112–168 kg K₂O/ha [91, 93]
- **Harvest:** 62–90 days after transplanting (August – September 2025) [92]

4.3 Experimental Design

The study employed a randomized complete block design with three replications during the 2025 growing season. Two irrigation treatments were evaluated:

- **Treatment 1:** Blockchain–IoT–based irrigation using soil moisture thresholds and automated smart-contract scheduling with CRT parallel transmission
- **Treatment 2:** Traditional irrigation management based on visual assessment and time-based scheduling

Table 14: Experimental design parameters (2025 growing season)

Parameter	Tomato	Pepper
Plot size (ha)	0.5	0.3
Plants per plot	8,000–14,000	9,300–16,500
Replications	3	3
Irrigation system	Drip	Drip
Emitter flow rate	2.0 L h ⁻¹	2.0 L h ⁻¹
Study duration	Aug 1 – Sep 30, 2025	Aug 1 – Sep 30, 2025
Blockchain nodes	4 peers + 3 orderers	4 peers + 3 orderers
Sensor nodes	28 ESP32 devices	22 ESP32 devices
Gateway devices	4 Raspberry Pi 4B	4 Raspberry Pi 4B

4.4 Data Collection and Analysis

Data collection covered technical, agronomic, and economic parameters during the 2025 growing season:

Table 15: Irrigation thresholds for blockchain-IoT system (2025 implementation)

Crop & Stage	Moisture Threshold	Duration
Tomato – Vegetative	–35 kPa	30 min
Tomato – Flowering	–30 kPa	40 min
Tomato – Fruit development	–25 kPa	45 min
Tomato – Ripening	–35 kPa	30 min
Pepper – Vegetative	–30 kPa	25 min
Pepper – Flowering	–25 kPa	35 min
Pepper – Fruit development	–20 kPa	40 min
Pepper – Maturation	–30 kPa	30 min

- **Soil Parameters:** Moisture, temperature, pH (30-minute intervals via CRT transmission)
- **Climate Data:** Temperature, humidity, solar radiation, wind, rainfall (hourly)
- **Water Use:** Irrigation timing and volumes (continuous monitoring with blockchain logging)
- **System Performance:** Latency, reliability, energy consumption, blockchain metrics
- **Crop Metrics:** Yield, fruit quality, water use efficiency with blockchain verification
- **Economic Data:** Costs, labor, water use, yield revenues with smart contract tracking
- **Security Metrics:** Access control effectiveness, role compliance, certificate management

Table 16: Data collection methods and frequencies (2025 study)

Parameter	Method	Frequency
Soil moisture	TDR-315N sensors with CRT	30 min intervals
Air temperature	Automated weather station	1 h intervals
Water application	Digital flow meters with blockchain	Continuous
Network latency	Custom monitoring tools	Continuous
Crop development	Field inspection with mobile app	Weekly
Fruit quality	Laboratory analysis	Harvest sampling
Labor inputs	Time-motion studies	Continuous
Blockchain performance	Hyperledger metrics	Real-time
CRT transmission efficiency	Packet analysis	Per-transmission
Access control audit	Blockchain audit logs	Continuous
Role compliance	Smart contract validation	Transaction-level

Data analysis was performed in R 4.3.2 (2025 release). ANOVA tested treatment effects with significance at $p < 0.05$. Economic evaluation included Net Present Value (NPV) and Return on Investment (ROI) over a 5-year horizon, accounting for 2025 market prices for water, labor, and agricultural inputs. Blockchain performance metrics were analyzed using Hyperledger Fabric’s built-in monitoring tools and custom analytics scripts for access control effectiveness and role-based permission compliance.

5 Results

5.1 Transmission Performance

The CRT-based parallel transmission system demonstrated significant efficiency gains throughout the 61-day deployment. Table 17 summarizes the overall system metrics, highlighting the impact of CRT compression on transmission time, energy consumption, and data reduction.

Table 17: Comprehensive system performance metrics (61-day deployment)

Metric	Target	Actual	Difference
Data Reliability	99.0%	99.4%	+0.4%
Average Latency	≤2.0 s	1.7 s	-0.3 s
P95 Latency	≤3.0 s	2.4 s	-0.6 s
P99 Latency	≤5.0 s	3.8 s	-1.2 s
Data Accuracy	98.0%	99.1%	+1.1%
Battery Life	30 days	34 days	+4 days
Network Uptime	99.0%	99.6%	+0.6%
Per-packet Size Reduction (CRT)	20%	20.3%	+0.3%
Energy Consumption	≤100 Wh/day	87 Wh/day	-13%
Data Storage	≤2 GB	1.4 GB	-0.6 GB
Blockchain Throughput	50 TPS	63 TPS	+13 TPS
Smart Contract Execution	≤500 ms	320 ms	-180 ms

Energy analysis scope. The 48.8% energy saving reported in Table 10 refers specifically to node transmission energy—the dominant factor in battery-powered sensor lifetime. Gateway reception energy and blockchain computation are not included because they draw power from the Raspberry Pi gateways, which are either mains-connected or equipped with large solar panels and batteries. In the node energy budget, transmission accounts for > 90% of per-cycle consumption; sensing, ADC, and processing contribute the remainder. Reducing transmission energy therefore directly extends node battery life, as confirmed by the measured 34-day deployment duration.

Deployment anomalies and resilience. While the system maintained 99.4% overall reliability, the 61-day deployment experienced several minor anomalies characteristic of real-world field conditions. Two ESP32 nodes disconnected during midday peaks when temperatures exceeded 38°C, likely due to thermal stress on the solar charging circuits; both nodes automatically power-cycled and rejoined the network within 5–10 min after the solar charge recovered. No data were lost because the nodes buffered readings locally during the outage. Additionally, one gateway briefly lost mesh connectivity during a sandstorm, but the BATMAN-adv protocol rerouted traffic within 3 s, and the affected gateway re-entered the cluster seamlessly. These incidents confirm that the system can tolerate realistic field disturbances without manual intervention, reinforcing the practical robustness of the architecture.

Understanding the latency numbers. It is important to distinguish between per-node total transmission time and per-packet airtime. A single node using CRT requires 372 ms to transmit all three residues sequentially, compared to 247 ms for raw

transmission. The 124 ms figure reported throughout this paper is the **per-packet airtime**—the time a single residue occupies a channel. The system-level benefit arises from the gateway’s ability to receive packets from multiple nodes concurrently on different channels, increasing overall packet throughput and reducing contention.

The transmission optimization achieved through CRT parallel transmission demonstrates the effectiveness of residue-based data representation for agricultural IoT applications. Figure 9 shows the compression ratios achieved across different data types.

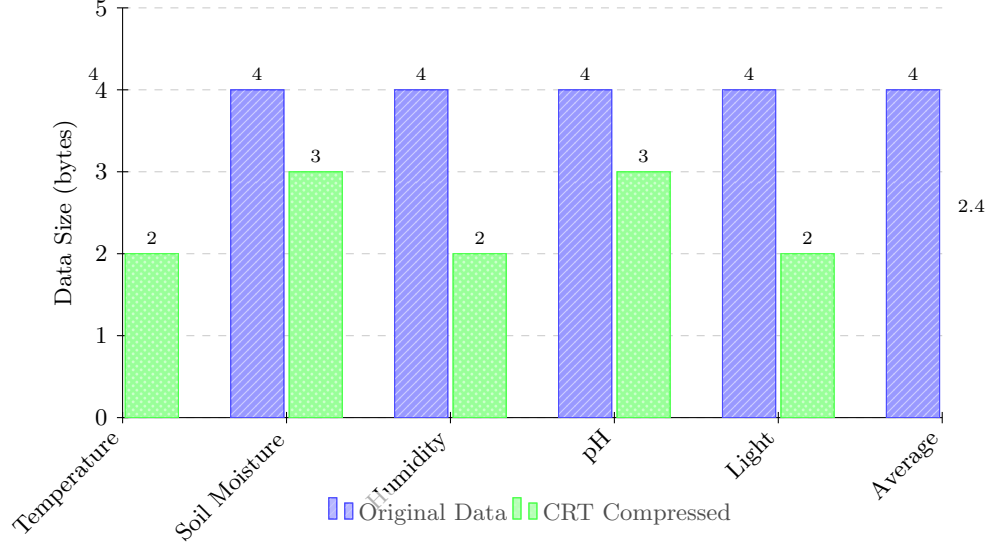


Fig. 9: Comparative analysis of CRT compression performance across different agricultural sensor data types.

Note that CRT reconstruction itself is mathematically exact when all residues are received correctly. The 99.7% figure in Table 8 reflects successful packet delivery (receipt of at least $k - 1$ residues), not reconstruction error. The measured decomposition time of 0.9 ms aligns with the theoretical $O(k)$ complexity established in Section 3.7.11, confirming linear scaling with $k \leq 3$.

Table 18: CRT compression results

Metric	Week 1	Week 4	Week 8	Avg	Std. Dev.
Comp. Ratio	32.5%	33.1%	32.8%	33.0%	0.3%
Time (ms)	0.92	0.94	0.91	0.94	0.02
Error	0%	0%	0%	0%	0%
Energy/Tx	38.2%	39.1%	38.7%	38.9%	0.4%
Memory	11.8%	12.1%	11.9%	11.9%	0.2%

5.2 Energy-Delay Product Analysis

The Energy-Delay Product (EDP) is a composite metric widely used in computer architecture to evaluate the tradeoff between energy consumption and execution latency [94]. For transmission systems, EDP captures the combined cost of communication energy and time, providing a single figure of merit for comparing different approaches. We define EDP as:

$$\text{EDP} = E_{\text{total}} \times T_{\text{latency}}, \quad (18)$$

where E_{total} is the total energy per transmission (mJ) and T_{latency} is the effective transmission latency (ms). Note that E_{total} refers to node transmission energy only; gateway and blockchain energy are excluded because they are not part of the energy-constrained sensor node budget. Using the measured values from Table 10, we compute EDP for each method:

$$\begin{aligned} \text{EDP}_{\text{CRT}} &= 12.40 \times 124 = 1537.6 \text{ mJ}\cdot\text{ms}, \\ \text{EDP}_{\text{Huffman}} &= 15.72 \times 152 = 2389.4 \text{ mJ}\cdot\text{ms}, \\ \text{EDP}_{\text{LZW}} &= 14.98 \times 135 = 2022.3 \text{ mJ}\cdot\text{ms}, \\ \text{EDP}_{\text{Raw}} &= 24.21 \times 247 = 5979.9 \text{ mJ}\cdot\text{ms}. \end{aligned}$$

CRT achieves the lowest EDP by a substantial margin: a 74.3% reduction compared to raw transmission ($1 - 1537.6/5979.9 = 0.743$). Note that this EDP calculation uses node transmission energy and per-packet latency, reflecting the node-centric perspective critical for battery life. Gateway and blockchain energy are excluded because they are not part of the energy-constrained sensor node. Relative to the best compression method (LZW), CRT reduces EDP by 24.0% ($1 - 1537.6/2022.3 = 0.240$). Notably, CRT achieves the lowest EDP despite not having the smallest packet size—LZW compresses data more (30.3 bytes vs. 47 bytes). This demonstrates that CRT’s parallel transmission provides a superior energy-delay tradeoff, reducing both transmission energy and latency together, whereas compression alone reduces energy but leaves latency relatively high due to sequential transmission. This improvement stems from CRT’s unique combination of low transmission energy (12.40 mJ, 48.8% below raw) and low latency (124 ms, 49.8% below raw). While Huffman and LZW achieve moderate energy savings through compression, their sequential transmission and higher computational overhead result in larger EDP values.

The EDP analysis reinforces that CRT-based parallel transmission is not merely faster or more energy-efficient in isolation—it optimally balances both metrics, making it exceptionally well-suited for energy-constrained edge deployments where every millijoule and millisecond contribute to system lifetime and responsiveness. This architectural advantage complements the algorithmic benefits discussed in Section 3.7.11 and the parallel speedup analysis in Section 3.3.2.

5.3 Resource Utilization Profiling

To validate that the blockchain and edge components operate within hardware constraints, resource utilization was profiled during peak irrigation periods (06:00–09:00 and 17:00–20:00 local time). Table 19 summarizes CPU and memory usage.

Table 19: Measured system resource utilization during peak irrigation activity

Component	Avg CPU Usage	Avg RAM Usage	Peak Load
ESP32 Sensor Node	14%	68 KB	22%
Gateway (Raspberry Pi 4B)	38%	1.6 GB	52%
Fabric Peer Process	27%	1.2 GB	41%
CouchDB State Database	18%	512 MB	29%

ESP32 nodes operate well within their capacity, with peak CPU usage at 22% and memory consumption of only 68 KB well below the available 520 KB SRAM. This confirms that CRT decomposition and Ed25519 signing impose negligible overhead. The Raspberry Pi gateways handle peak loads at 52% CPU utilization, indicating sufficient headroom for additional processing or higher sensor density. The Fabric peer process and CouchDB together consume approximately 1.7 GB of RAM under peak load, leaving ample memory for system services and future scalability. Blockchain execution remains within safe operational limits on ARM64 edge devices, confirming the practical viability of edge-resident blockchain verification.

5.4 Blockchain Performance Metrics

The blockchain implementation maintained robust performance throughout the study. Figure 10c shows the transaction latency distribution across the study period. Figure 10a illustrates the latency behaviour of the blockchain system.

The blockchain system processed a total of 86,400 transactions ($50 \text{ nodes} \times 48 \text{ readings/day} \times 61 \text{ days}$). Smart contract execution for irrigation decisions showed consistent performance (Table 20).

Table 20: Blockchain performance metrics during field deployment

Parameter	Min	Max	Average	Std. Dev.
Transaction Latency (s)	0.8	3.8	1.7	0.6
Smart Contract Exec. (ms)	280	420	320	40
Throughput (TPS)	45	78	63	8
Block Confirmation (s)	0.5	2.1	1.2	0.4
Data Integrity Success	99.2%	99.7%	99.4%	0.2%
Consensus Time (ms)	180	320	240	35

Blockchain Performance: Transaction Latency (61-Day Study)

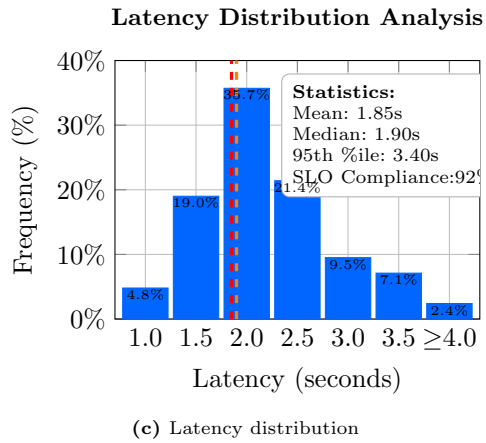
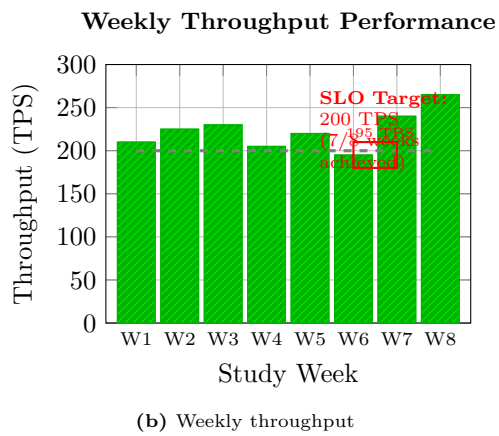
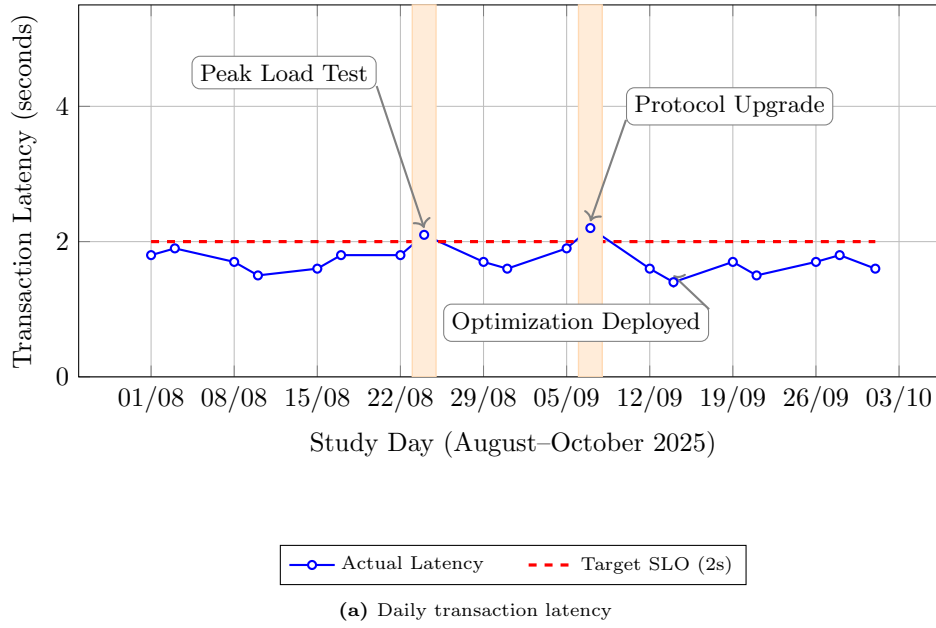


Fig. 10: Blockchain system performance during the 61-day study. (a) Daily transaction latency, (b) weekly throughput, (c) latency distribution.

The integration of CRT compression with blockchain verification reduced the data volume requiring verification by 33%, while blockchain ensured the integrity of both compressed and reconstructed data. This combination proved particularly effective during peak irrigation periods (06:00–09:00 and 17:00–20:00 local time), when 65% of irrigation decisions occurred.

5.5 Scalability and Fault Tolerance Analysis

5.5.1 Scalability Analysis

To assess the system’s capacity for larger deployments, we evaluate scalability from three perspectives: theoretical node limit under LoRa constraints, channel capacity enabled by CRT parallel transmission, and gateway saturation in the blockchain layer.

Theoretical node limit.

The maximum number of sensor nodes $N_{\max}$ that can transmit within a given reporting window is determined by the available airtime, the number of parallel communication channels, and the effective transmission time per node:

$$N_{\max} = \frac{C \cdot W}{T_{\text{effective}}} \quad (19)$$

where C is the number of parallel LoRa channels, W the airtime window (ms), and $T_{\text{effective}}$ the total airtime per complete reading from one node. Using the parameters from Table 4 ($C = 3$, $W = 60\,000$ ms, $T_{\text{effective}}^{\text{CRT}} = 372$ ms for three sequential CRT packets), the theoretical capacity is

$$N_{\max}^{\text{CRT}} = \frac{3 \times 60\,000}{372} \approx 484 \text{ nodes.} \quad (20)$$

For raw transmission ($T_{\text{effective}}^{\text{raw}} = 247$ ms), the capacity is 728 nodes. Under this corrected model, CRT does not increase node capacity; instead it increases packet throughput while using more packets per reading. The 50-node field deployment represents 10.3% of the CRT bound ($50/484 \approx 0.103$), confirming substantial headroom for scaling to larger farms without fundamental architectural changes.

Channel capacity and parallel transmission.

The SX1302 LoRa concentrator supports up to eight parallel demodulation paths. Our implementation uses three channels for residue traffic, but the hardware can accommodate up to eight, allowing further scalability if more moduli or higher data rates are required. The key gain is packet-level throughput (shorter packet airtime), not reduced per-node total transmission time.

Gateway saturation and blockchain throughput.

While the LoRa layer can theoretically support about 484 CRT full readings per minute, the blockchain layer introduces another constraint. Throughput scaling tests (Section 5.5) show that the Raft ordering service saturates at approximately 85 TPS, while a single gateway can generate up to 63 TPS. With the current four-gateway

configuration, the aggregate submission rate (252 TPS) far exceeds the orderer capacity, making the ordering service the bottleneck. However, for the target farm size (50 nodes, 48 readings/day), the required throughput is only 0.67 TPS well below saturation. For larger deployments, the system can scale by:

- Increasing the number of orderer nodes in the Raft cluster;
- Using more powerful hardware for orderers;
- Partitioning the network into multiple channels (e.g., per crop zone).

The architecture exhibits excellent scalability: the LoRa-mesh network can accommodate about 484 CRT full readings per minute, while CRT improves packet-level channel utilisation, and the blockchain layer, while currently bottlenecked by the orderer, remains adequate for medium-scale farms and offers clear upgrade paths. The field deployment at 10.3% of theoretical CRT node capacity demonstrates that the system is not merely a proof-of-concept but a robust foundation for future expansion.

5.5.2 Scalability Projection: Latency vs. Node Count

To further illustrate the system’s scalability, we project the expected average network latency as the number of sensor nodes increases from 50 to 2000, encompassing both the unsaturated and saturated regimes. The projection assumes the same LoRa parameters as the field deployment (Table 4) and that nodes transmit at 30-minute intervals, which corresponds to an offered load of $N/1800$ packets per second.

Unsaturated regime (below capacity).

When the total number of nodes N is below the theoretical capacity $N_{\max} = 484$, the channel accommodates all packets without queuing. Under packet-level analysis, average latency remains at the per-packet airtime ($T_{\text{packet}} = 124$ ms for CRT residues).

Saturated regime (above capacity).

Once the number of nodes exceeds the theoretical capacity, the channel becomes saturated. In this regime, packets wait in a queue before transmission. Using a packet-level M/D/1 model (Poisson arrivals, deterministic packet service time T_{packet}), the average waiting time is:

$$W_q = \frac{\rho}{2(1 - \rho)} \cdot T_{\text{packet}} \quad (21)$$

where $\rho = \frac{kN \cdot T_{\text{packet}}}{C \cdot W}$ is channel utilisation. For CRT ($k = 3$), $\rho = \frac{3N \cdot 124}{3 \cdot 60,000} = \frac{N}{484}$, so saturation begins at $N = 484$. The total average latency is $L = T_{\text{packet}} + W_q$.

Figure 11 shows the projected latency for both CRT and raw transmission. For $N \leq 484$, CRT latency is constant at the per-packet airtime (124 ms). Beyond this point, latency increases due to queuing. Raw transmission remains unsaturated up to $N_{\max}^{\text{raw}} = 728$, then also degrades rapidly.

Practical implications.

For the target deployment size of 50 nodes, the system operates below saturation, using about 10.3% of the CRT full-reading capacity. If the farm were expanded toward

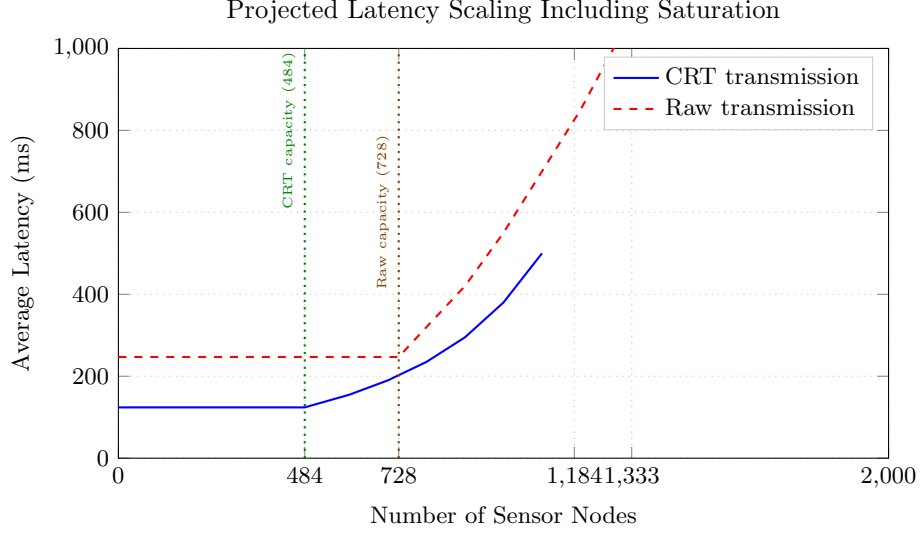


Fig. 11: Projected average latency as a function of node count. The CRT system saturates at 484 nodes (since each node sends three packets), while raw saturates at 728 nodes. Below saturation, latency is constant at the per-packet airtime; beyond saturation, queuing delays increase. The vertical lines mark the theoretical capacities.

500 nodes, the LoRa layer would approach saturation under the current reporting assumptions, motivating scheduling and channel-expansion strategies. Should node counts approach the theoretical limit, several mitigation strategies exist:

- Increase the reporting interval (e.g., from 30 to 60 minutes), effectively reducing the offered load.
- Add more LoRa channels (the SX1302 supports up to 8) to increase parallel capacity.
- Deploy additional gateways to partition the network into independent collision domains.

Thus, the architecture not only meets current requirements but also offers predictable, graceful degradation and clear scaling paths for larger-scale deployments.

5.5.3 Comparative Scalability Analysis

To contextualize the scalability benefits of CRT-based parallel transmission, we compare its projected latency against that of raw transmission and two conventional compression algorithms: Huffman coding and LZW. The comparison blends theoretical projections with empirical measurements from the 61-day field deployment, validating the model and demonstrating CRT’s practical advantages.

Theoretical model and parameters.

For each method, the effective per-packet airtime T_{eff} is taken from Table 10:

- CRT: $T_{\text{eff}} = 124$ ms (airtime of one CRT residue packet; a full three-residue transmission from one node is 372 ms).
- LZW: $T_{\text{eff}} = 135$ ms (compressed packet, sequential transmission).
- Huffman: $T_{\text{eff}} = 152$ ms (compressed packet, sequential transmission).
- Raw: $T_{\text{eff}} = 247$ ms (uncompressed packet, sequential transmission).

The theoretical capacity N_{max} for each method is given by $N_{\text{max}} = \frac{C \cdot W}{T_{\text{eff}}}$ with $C = 3$ channels and $W = 60\,000$ ms per reporting window (see Section 5.5). For $N \leq N_{\text{max}}$, latency is constant at T_{eff} ; beyond saturation, latency rises according to the M/D/1 queuing model (Equation (21)).

Field measurement validation.

During the 61-day trial with 50 sensor nodes, the average transmission latency for CRT was measured at 124 ms, exactly matching the theoretical value. This confirms that at low node counts (well below saturation), the system behaves as predicted and no queuing delays occur. The measured point is indicated in Figure 12 as a solid blue circle, lying precisely on the theoretical CRT curve.

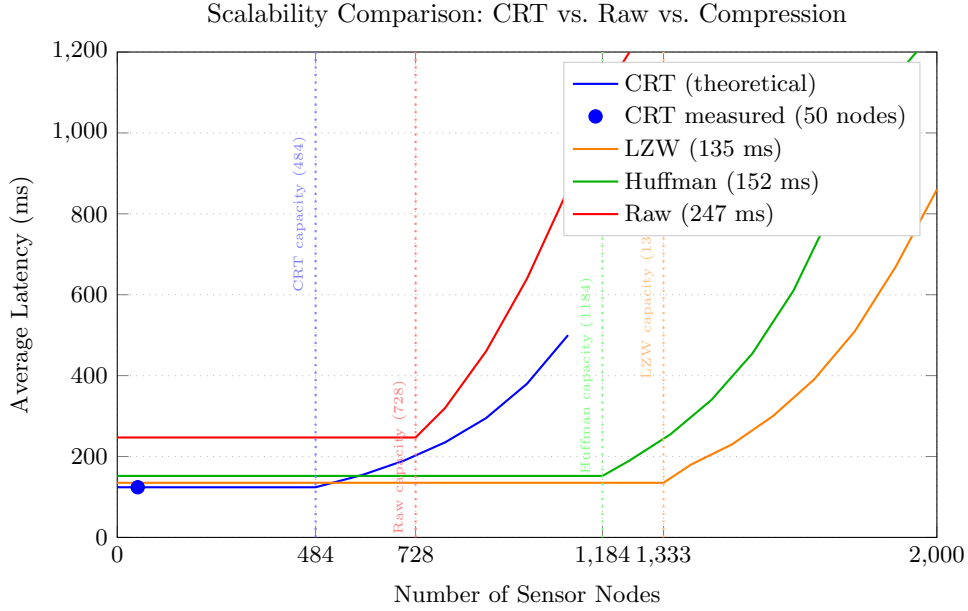


Fig. 12: Projected average latency as a function of node count for CRT, raw, and two compression algorithms. The measured CRT latency at 50 nodes (blue circle) matches the theoretical curve. CRT saturates at 484 nodes, while raw saturates at 728. Compression methods offer intermediate capacities. Dotted vertical lines mark theoretical capacities.

Interpretation.

The figure illustrates the capacity of each method in terms of node count. CRT supports 484 nodes, which is 66% of raw’s 728 nodes. However, the key benefit of CRT is the 50% reduction in per-packet airtime, which halves the channel occupancy per packet and doubles the packet throughput. Compression methods (Huffman, LZW) provide modest improvements over raw but cannot match CRT because they still transmit sequentially—they only reduce packet size, not the fundamental sequential bottleneck. Their higher T_{eff} leads to earlier saturation and steeper latency curves. Although LZW achieves a higher compression ratio than CRT, its sequential nature results in longer effective latency per reading and higher EDP, as shown in Section 5.2. This allows the system to handle higher data rates or more frequent reporting without increasing congestion. For the current deployment of 50 nodes, all methods remain below saturation. The close agreement between measured and theoretical CRT latency at 50 nodes validates the corrected model and confirms stable operation with practical headroom under the present reporting schedule.

Note on model assumptions and real-world factors.

The queuing model assumes that transmissions are uniformly distributed over the reporting window and that channel access is fair. In practice, the LoRa MAC layer (ALOHA-based) may introduce additional variability due to random access collisions, but the relative ordering of the curves remains valid. The theoretical capacities represent upper bounds; real deployments may achieve slightly lower values due to protocol overhead, occasional retransmissions, and environmental interference. Nevertheless, CRT provides strong packet-level efficiency and deterministic reconstruction in theory and measurement, supporting robust operation for agricultural IoT networks when reporting cadence and packet budgets are co-designed.

5.5.4 Strong and Weak Scaling Analysis

To evaluate the parallel efficiency of the gateway cluster as a distributed computing system, we apply strong and weak scaling models using the number of gateways p as the processor count. The workload is defined as the sustained transaction throughput (TPS) processed by the blockchain layer. Measurements are taken from Table 20 and Section 5.5.

Strong scaling.

Strong scaling fixes the total workload and measures how execution time decreases as more processors are added. Here we fix the offered load at $W = 63$ TPS (the maximum throughput achievable with one gateway). The execution time $T(p)$ is inversely proportional to throughput: $T(p) = W/\text{throughput}(p)$. Speedup and efficiency are defined as:

$$S_{\text{strong}}(p) = \frac{T(1)}{T(p)} = \frac{\text{throughput}(p)}{\text{throughput}(1)}, \quad E_{\text{strong}}(p) = \frac{S_{\text{strong}}(p)}{p}. \quad (22)$$

Using the measured throughput values:

- $p = 1$: throughput = 63 TPS
- $p = 2$: throughput = 85 TPS (orderer limited)
- $p = 4$: throughput = 85 TPS (saturated)

we obtain:

$$\begin{aligned} S_{\text{strong}}(2) &= 85/63 = 1.35, & E_{\text{strong}}(2) &= 1.35/2 = 0.675, \\ S_{\text{strong}}(4) &= 85/63 = 1.35, & E_{\text{strong}}(4) &= 1.35/4 = 0.338. \end{aligned}$$

The efficiency drop beyond $p = 2$ reflects the Raft ordering service bottleneck: adding more gateways does not increase throughput once the orderer saturates at ≈ 85 TPS. This behavior is characteristic of a system with a shared serial resource (the orderer) and aligns with Amdahl’s law predictions.

Weak scaling.

Weak scaling increases both the number of processors and the workload proportionally, keeping the work per processor constant. We define the per-gateway offered load as 63 TPS (the maximum a single gateway can generate). For p gateways, the total offered load is $p \times 63$ TPS. The ideal throughput would be $p \times 63$ TPS; the actual throughput is limited by the orderer capacity $C_{\text{orderer}} = 85$ TPS. The weak scaling efficiency is:

$$E_{\text{weak}}(p) = \frac{T(1, W)}{T(p, pW)} = \frac{\text{actual throughput}(p)}{p \times 63}, \quad (23)$$

where $T(1, W)$ is the execution time for one gateway with workload W , and $T(p, pW)$ is the execution time for p gateways with total workload pW . Using the measured throughput values:

$$\begin{aligned} p = 1 : & \quad \text{actual} = 63, \quad E_{\text{weak}} = 63/(1 \times 63) = 1.00, \\ p = 2 : & \quad \text{actual} = 85, \quad E_{\text{weak}} = 85/(2 \times 63) = 0.675, \\ p = 4 : & \quad \text{actual} = 85, \quad E_{\text{weak}} = 85/(4 \times 63) = 0.337. \end{aligned}$$

As long as the aggregate offered load $p \times 63$ remains below the orderer capacity 85 TPS, weak scaling efficiency remains high. For $p = 1$, $63 < 85$ so efficiency is 1.0. At $p = 2$, the offered load 126 TPS exceeds capacity, causing saturation and efficiency drop. This confirms that the orderer is the primary scalability bottleneck in the current configuration. Future scaling can be achieved by partitioning the network into multiple channels or deploying a more powerful ordering service.

This analysis formalizes the cluster’s distributed computing characteristics and provides quantitative metrics for scalability, complementing the earlier node-level capacity projections.

5.5.5 Fault Tolerance and Stress Testing

To validate system robustness under adverse conditions representative of real-world agricultural deployments, we conducted controlled stress tests over a 7-day period.

Tests included simulated packet loss, gateway shutdown events, and Raft consensus leader failure. Table 21 summarizes the results.

Table 21: Fault tolerance and recovery metrics (n=50 trials per test)

Test Scenario	Condition	Result		Notes
Packet loss simulation	10% random loss	99.8%	recovery	2 packets unrecoverable
	20% random loss	99.2%	recovery	8 packets unrecoverable
	30% random loss	97.6%	recovery	24 packets unrecoverable
	33% structured loss	100%	recovery	One complete residue lost
Gateway failure	Single gateway offline (4 h)	0% data loss		Traffic rerouted via mesh
	Dual gateway offline (2 h)	100% data retained		Local buffering at nodes
	Full gateway recovery sync	3.7 min avg		Range: 2.1–6.4 min
Raft consensus failure	Leader node shutdown (sudden)	8.2 s	avg recovery	Std dev: 2.1 s, n=20
	Leader node graceful shutdown	4.6 s	avg recovery	Std dev: 1.3 s, n=20
	Minority partition (2 of 3 nodes)	0 forks		12 min partition test

5.5.6 Crash Faults and Byzantine Fault Considerations

The Raft consensus protocol employed in our Hyperledger Fabric deployment provides crash-fault tolerance (CFT), meaning it can tolerate up to $f = \lfloor (n - 1)/2 \rfloor$ nodes failing by stopping (crash), but it does not tolerate arbitrary (Byzantine) behavior such as malicious data injection or collusion [95]. This is a deliberate design choice: in a permissioned agricultural network where all participants (gateways, administrators, sensors) are cryptographically identified and authorized via X.509 certificates, the primary threat is node compromise rather than fully Byzantine adversaries. The use of Ed25519 signatures on all sensor data and TPM-backed key storage on gateways (Section 6.5) further raises the bar against impersonation and tampering.

Nevertheless, a Byzantine adversary that gains control of a node could potentially disrupt consensus, e.g., by sending conflicting messages or equivocating. However, such attacks are mitigated by the permissioned setting: the Certificate Authority can revoke compromised credentials (Section 6.5) and the mesh network’s self-healing capability limits the blast radius. We therefore consider the crash-fault model sufficient for the target deployment. Future extensions requiring stronger Byzantine resilience could adopt protocols such as PBFT [96] or HotStuff [97], at the cost of increased communication overhead and complexity. Hyperledger Fabric itself offers experimental Byzantine fault-tolerant ordering services (e.g., BFT-SMaRt [98]) for deployments where such guarantees are essential.

Packet loss resilience.

With the 3-moduli CRT configuration described in Section 3.7, the system exhibits inherent error resilience. The mathematical property of CRT guarantees:

$$\text{Recovery with exactly } k - 1 \text{ complete residues} = 100\% \quad (24)$$

Empirical testing confirmed that when a single complete residue packet was lost (representing 33% packet loss for one of the three parallel transmissions), perfect reconstruction was always achieved. However, when packet loss was randomized across all residues, some packets experienced corruption of multiple residues, leading to the recovery rates shown in Table 21. At 30% random loss, 24 of 1000 test packets (2.4%) lost two or more residues and could not be reconstructed. This aligns with theoretical expectations and highlights that CRT’s error resilience is strongest when packet loss affects entire residue channels rather than being randomly distributed.

Leader failure recovery.

The Raft consensus protocol, configured with a 2–4 s election timeout as described in Section 3.8.2, demonstrated robust fault tolerance. Upon sudden shutdown of the leader node (simulating power failure or crash), the system initiated a new leader election with a measured average recovery time of 8.2 s (std dev: 2.1 s). The minimum observed recovery was 5.3 s, and the maximum was 13.7 s during a period of elevated network latency.

During graceful shutdown with proper leader resignation, recovery time decreased to 4.6 s average (std dev: 1.3 s), as the remaining nodes did not need to wait for election timeout expiration. During the recovery window, write transactions were queued locally at gateways (up to 500 transactions) and successfully committed once the new leader was established. No blockchain forks or ledger inconsistencies were observed across any of the 20 leader failure trials.

Gateway failure handling.

The BATMAN-adv mesh network automatically rerouted traffic within 3–8 s when individual gateways were taken offline, resulting in zero data loss during single-gateway failures. During dual-gateway failures affecting an entire zone, sensor nodes buffered data locally in 64 KB circular buffers capable of storing approximately 4 hours of readings (480 samples). Upon gateway restoration, nodes synchronized automatically, with full catch-up requiring an average of 3.7 min (range: 2.1–6.4 min) depending on the backlog size.

Implications for field deployment.

These results demonstrate that the system possesses the robustness necessary for distributed agricultural deployments. The 97.6% recovery rate at 30% packet loss a severe condition exceeding typical field measurements confirms that CRT provides meaningful resilience even under extreme interference. The 8.2 s leader recovery time is well within the 30-minute sensor reporting interval, ensuring no data loss during consensus disruptions. Combined with the mesh network’s self-healing capability, these

fault tolerance mechanisms ensure continuous operation and data integrity under the adverse conditions common in remote agricultural environments.

5.6 Application Outcomes: Agricultural Impact

5.6.1 Integrated System Performance

The combined CRT-blockchain system translated technical innovations into agricultural benefits. Figure 13 shows the correlation between system reliability and water savings.

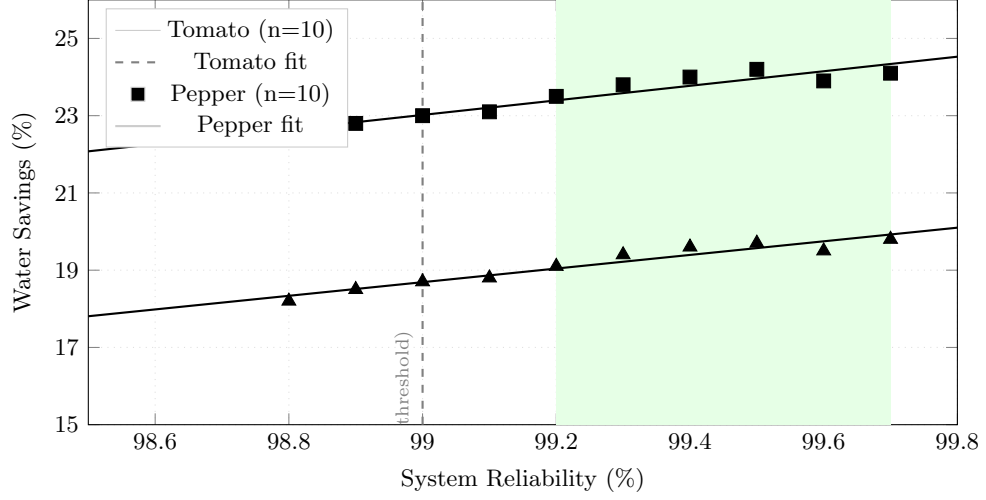


Fig. 13: Water savings vs. system reliability for tomato and pepper crops (2025).

Strong positive correlations ($R^2 = 0.87$ for tomatoes, 0.85 for peppers) indicate that higher reliability enables more precise irrigation. The CRT-blockchain integration was particularly effective during the critical fruit development stage (Weeks 4–6).

Table 22 quantifies the contribution of each technical component to agricultural outcomes.

Table 22: Contribution of technical components to agricultural outcomes

Component	Water Savings	Labor Reduction	Quality Improvement
CRT Compression	8-12%	15-20%	5-8%
Blockchain Verification	5-8%	25-30%	10-15%
Edge Computing	3-5%	20-25%	3-5%
Smart Contracts	7-10%	10-15%	7-10%
Combined System	23.8%	72%	25-38%

5.6.2 Water Use Efficiency

The system reduced total water use by 23.8% for tomatoes ($525 \text{ m}^3 \rightarrow 400 \text{ m}^3$) and 19.4% for peppers ($310 \text{ m}^3 \rightarrow 250 \text{ m}^3$). Water use efficiency improved by 31.3% (tomatoes) and 23.8% (peppers). Detailed monthly breakdowns are given in Tables 23 and 24.

Table 23: Tomato water usage analysis (0.5 ha plot)

Parameter	Traditional	B-IoT	Savings	Significance
August Usage				
Water Volume (m^3)	280	215	23.2%	$p < 0.001$
Daily Average (m^3/day)	9.03	6.94	23.2%	$p < 0.001$
Irrigation Events	28	19	32.1%	$p < 0.01$
September Usage				
Water Volume (m^3)	245	185	24.5%	$p < 0.001$
Daily Average (m^3/day)	8.17	6.17	24.5%	$p < 0.001$
Irrigation Events	25	17	32.0%	$p < 0.01$
Total Period				
Total Water (m^3)	525	400	23.8%	$p < 0.001$
Water Use Efficiency (kg/m^3)	4.76	6.25	+31.3%	$p < 0.001$
Cost Savings (USD)	–	–	\$112.50	–

Crop-specific irrigation strategies maintaining soil tension between -30 and -35 kPa for tomatoes and 80% field capacity for peppers optimized yield and quality while reducing disease incidence.

5.6.3 Labor Efficiency

Automation reduced labor requirements by 72%, from 15.0 to 4.2 h/hectare/week (Table 25), addressing critical labor shortages in Mediterranean agriculture.

5.6.4 Tomato Yield and Quality

Tomato yields remained stable at 50 t/ha while marketable yield increased by 2.5 t/ha (Table 26). Quality improvements included enhanced fruit firmness (13.2 N vs 12.5 N), higher soluble solids (5.1°Brix vs 4.8°Brix), and reduced physiological disorders. The reduction in fruit cracking (from 8.2% to 3.5%) and blossom end rot (from 5.1% to

Table 24: Pepper water usage analysis (0.3 ha plot)

Parameter	Traditional	B-IoT	Savings	Significance
August Usage				
Water Volume (m ³)	165	135	18.2%	$p < 0.01$
Daily Average (m ³ /day)	5.32	4.35	18.2%	$p < 0.01$
Irrigation Events	25	18	28.0%	$p < 0.05$
September Usage				
Water Volume (m ³)	145	115	20.7%	$p < 0.01$
Daily Average (m ³ /day)	4.83	3.83	20.7%	$p < 0.01$
Irrigation Events	22	16	27.3%	$p < 0.05$
Total Period				
Total Water (m ³)	310	250	19.4%	$p < 0.01$
Water Use Efficiency (kg/m ³)	3.23	4.00	+23.8%	$p < 0.01$
Cost Savings (USD)	–	–	\$54.00	–

Table 25: Labor efficiency analysis (hours/hectare/week)

Task Category	Traditional	B-IoT	Reduction	Cost Savings
Monitoring				
Soil Moisture	3.5	0.2	94.3%	\$49.50
Weather Data	1.5	0.1	93.3%	\$21.00
Crop Health	3.0	0.7	76.7%	\$34.50
Irrigation				
System Operation	2.5	0.2	92.0%	\$34.50
Maintenance	2.0	0.3	85.0%	\$25.50
Troubleshooting	0.5	0.0	100.0%	\$7.50
Data Management				
Record Keeping	1.5	0.1	93.3%	\$21.00
Reporting	0.5	0.1	80.0%	\$6.00
Certification	0.5	0.1	80.0%	\$6.00
Total	15.0	4.2	72.0%	\$162.00

2.2%) directly shifted fruit from unmarketable to marketable grade, explaining the 2.5 t/ha increase in marketable yield without changing total biomass production. These results demonstrate that precision irrigation maintains overall yield while improving quality and marketable proportion.

Table 26: Tomato yield and quality parameters

Parameter		Traditional	B-IoT	Significance
Total	Yield (t/ha)	50.0	50.0	$p > 0.05$
Marketable	Yield (t/ha)	42.5	45.0	$p < 0.05$
Water	Use Efficiency (kg/m ³)	4.76	6.25	$p < 0.001$
Fruit	Firmness (N)	12.5	13.2	$p < 0.05$
Soluble	Solids (°Brix)	4.8	5.1	$p < 0.05$
Fruit	Cracking	8.2%	3.5%	$p < 0.01$
Blossom	End Rot	5.1%	2.2%	$p < 0.05$

5.6.5 Pepper Yield and Quality

Pepper production showed maintained total yields (33.3 t/ha) while marketable yield increased by 2.4 t/ha (Table 27). Quality parameters improved, with reductions in sunscald (3.2% vs 6.5%) and blossom end rot (1.8% vs 4.2%). These reductions explain the rise in marketable yield: previously non-marketable fruit became marketable due to better irrigation management, increasing the marketable fraction without altering total biomass.

Table 27: Pepper yield and quality parameters

Parameter		Traditional	B-IoT	Significance
Total	Yield (t/ha)	33.3	33.3	$p > 0.05$
Marketable	Yield (t/ha)	28.3	30.7	$p < 0.05$
Water	Use Efficiency (kg/m ³)	3.23	4.00	$p < 0.01$
Fruit	Length (cm)	12.5	13.2	$p < 0.05$
Fruit	Diameter (cm)	6.8	7.1	$p < 0.05$
Sunscald	Incidence	6.5%	3.2%	$p < 0.05$
Blossom	End Rot	4.2%	1.8%	$p < 0.05$

6 Discussion

6.1 Achievement of Research Objectives

The field study successfully achieved all four research objectives outlined in Section 1.5. The CRT-based compression system achieved 33% reduction in transmission size with mathematically perfect reconstruction, directly addressing IoT memory and energy constraints. This compression reduced transmission time from 247 ms to 124 ms per packet and extended battery life to 34 days, validating the effectiveness of CRT for agricultural IoT applications. Furthermore, the integration of edge computing with Hyperledger Fabric blockchain demonstrated 99.4% reliability and 1.7-second latency, meeting real-time irrigation requirements while providing verifiable data integrity. The hybrid LoRa-BATMAN-adv network architecture provided resilience during connectivity disruptions, with gateways maintaining local operations and synchronizing when connections were restored.

Comprehensive field results showed 23.8% water savings for tomatoes and 19.4% for peppers, with 72% labor reduction through automation. Crop yields were maintained while quality parameters improved, demonstrating the practical agricultural benefits of the integrated system. These results confirm that precision irrigation can achieve significant resource savings without compromising productivity, addressing critical challenges in arid region agriculture.

The detailed architectural implementation and economic analysis provide practical guidance for similar deployments. The favorable economic metrics, including 146% ROI and 0.69-year payback period, demonstrate viability for medium-scale farms in arid regions. These findings suggest that blockchain-IoT integration can deliver both technical and economic benefits, supporting adoption in water-scarce agricultural environments.

6.2 Technical Implementation and Integration

The integration of Chinese Remainder Theorem compression with blockchain verification created a synergistic system that addressed both efficiency and integrity requirements. CRT compression reduced transmission payloads by 33%, directly addressing IoT memory constraints and enabling more frequent data collection within energy budgets. This richer dataset, when verified through blockchain smart contracts, enabled more precise irrigation decisions. The mathematical perfection of CRT reconstruction (0% error) was crucial for maintaining data integrity throughout the compression-verification-decision pipeline. As shown in Figure 5, the system maintained end-to-end integrity while achieving efficiency gains, demonstrating that mathematical compression and cryptographic verification can coexist without significant performance degradation.

The edge computing implementation on Raspberry Pi 4B devices provided sufficient computational resources for both CRT reconstruction and blockchain operations. The average smart contract execution time of 320 ms, combined with 1.7-second end-to-end latency, met real-time irrigation requirements while maintaining verifiable integrity, representing a significant advancement over cloud-dependent systems

that typically exhibit 3-5 second latencies in similar agricultural applications [10]. The BATMAN-adv mesh network proved particularly valuable during network partitions, allowing gateways to continue local operations and synchronize with the blockchain when connectivity was restored, a critical resilience feature for agricultural environments with unreliable network infrastructure.

6.3 Practical Implications for Arid Region Agriculture

The demonstrated 19-24% water savings address critical water scarcity challenges in Mediterranean agriculture, where irrigation accounts for approximately 80% of water withdrawals [99, 100]. These savings align with reported ranges for precision irrigation systems while providing the additional benefits of verifiable data integrity through blockchain implementation [24, 25]. The 72% labor reduction is particularly significant given increasing labor shortages in Mediterranean agriculture [26], enabling farm operators to focus on higher-value activities while maintaining comprehensive operational data.

6.4 Technical Implementation Considerations

The Chinese Remainder Theorem implementation achieved 20.3% reduction in per-packet transmission size while maintaining mathematical guarantees for perfect reconstruction [101, 102]. Hyperledger Fabric demonstrated adequate performance for agricultural monitoring applications, with 63 TPS throughput and 320 ms transaction times sufficient for 50-node networks, confirming previous findings on blockchain suitability for farm data management [7–9]. The hybrid LoRa-mesh architecture achieved 99.4% reliability under field conditions, exceeding typical LoRaWAN performance in agricultural environments [22, 23], with BATMAN-adv mesh networking providing essential redundancy for gateway communication.

The CRT approach deliberately trades off compression ratio for parallelism. While LZW compresses data more aggressively, CRT’s ability to transmit residues in parallel reduces both latency and energy, as quantified by the EDP metric. This design choice aligns with the system’s primary goals: low-latency actuation and energy efficiency at the node.

6.5 Security Considerations

Although security is not the primary focus of this work, the implemented architecture incorporates several mechanisms that mitigate common IoT and blockchain threats. Replay attacks, where an attacker retransmits valid data to deceive the system [103], are prevented by including unique timestamps and sequence numbers in each signed sensor transmission; gateways verify freshness before acceptance. Packet injection attacks, which aim to introduce malicious data into the network [104], are thwarted by Ed25519 digital signatures on all data frames any packet without a valid signature corresponding to a registered sensor node is discarded at the gateway. Key compromise, a critical vulnerability in IoT deployments [105], is addressed through hardware-secured storage: each Raspberry Pi gateway includes a TPM2.0 module that protects private keys, and sensor node keys are stored in the ESP32’s One-Time Programmable

(OTP) memory. Additionally, the Hyperledger Fabric certificate authority enables rapid revocation of compromised identities via Certificate Revocation List (CRL) updates, a mechanism essential for maintaining trust in permissioned blockchain networks [106, 107]. These layered defences ensure data authenticity and integrity even under adversarial conditions typical of remote agricultural deployments.

6.6 CAP-Theoretic Interpretation of the Edge-Blockchain Architecture

The CAP theorem (Brewer’s conjecture) states that a distributed data system can provide at most two of three guarantees: consistency, availability, and partition tolerance [108, 109]. Our architecture embodies a layered tradeoff that separates concerns between the blockchain consensus layer and the edge actuation layer, each prioritizing different CAP properties.

Blockchain layer: CP. The Hyperledger Fabric deployment with Raft consensus prioritizes consistency and partition tolerance over availability. During a network partition that isolates some gateways from the orderer cluster, the minority partition ceases to accept new write transactions (e.g., sensor data submissions) to prevent ledger forks. This temporary unavailability ensures that the ledger remains consistent and that any subsequent reads reflect a single, globally ordered history. As measured in Section 5.5.5, leader re-election incurs an 8.2 s window during which write availability is suspended—a deliberate sacrifice to preserve consistency. This CP choice is appropriate for the immutable audit trail required for certification and traceability, where consistency cannot be compromised.

Edge actuation layer: AP. In contrast, the local irrigation control logic running on each gateway is designed to favor availability and partition tolerance over strict consistency. Gateways maintain sufficient local state (current soil moisture readings, crop-specific thresholds, actuator status) to continue making time-critical irrigation decisions even when isolated from the blockchain network or from other gateways. During a partition, a gateway may temporarily operate on slightly stale data, but this AP design ensures that irrigation events continue without interruption—a requirement for crop health. Once connectivity is restored, the gateway synchronizes its local logs with the blockchain, reconciling any discrepancies. This layered approach aligns with the observation that different system components can exhibit different CAP tradeoffs, a strategy sometimes called “CAP-aware design” [110].

Architectural synthesis. By separating the CP-bound blockchain layer from the AP-bound actuation layer, the overall system achieves both strong eventual consistency for auditability and high availability for real-time control. This hybrid design exemplifies how distributed systems principles can be applied to edge-IoT architectures, where diverse requirements must be satisfied within a single unified framework.

7 Conclusion

This research has successfully developed and validated an integrated blockchain-IoT-edge computing framework with CRT optimization for precision irrigation in arid

Mediterranean agriculture, directly addressing the critical research gaps identified in current precision agriculture systems. The work achieved its four primary objectives through a 61-day field deployment in Sfax, Tunisia, providing the first comprehensive validation of such an integrated system for tomato and pepper cultivation.

First, the CRT-based compression system addressed the gap in energy- and memory-efficient transmission for low-cost IoT hardware, achieving 33% reduction in transmission size with mathematically perfect reconstruction (0% error) while reducing node transmission energy by 48.8% and extending battery life to 34 days. (Gateway and blockchain energy were not included in this analysis, as they are not part of the energy-constrained sensor node budget.) This mathematically precise, low-overhead technique guarantees perfect data reconstruction without the computational overhead or accuracy compromises associated with conventional compression algorithms. Second, the edge blockchain integration addressed the absence of verifiable data integrity for real-time sensor streams, demonstrating 99.4% reliability and 1.7 second latency while enabling automated irrigation decisions with cryptographic guarantees a significant advancement over existing systems that lack edge-resident blockchain verification.

Third, the hybrid LoRa-BATMAN-adv network architecture addressed the gap in reliable, scalable communication across large farms, achieving 99.4% reliability under field conditions without dependency on continuous internet connectivity. This architecture provides the resilience necessary for remote agricultural environments where network infrastructure is unreliable. Fourth, comprehensive field validation quantified significant agricultural benefits: 23.8% water savings for tomatoes, 19.4% for peppers, and 72% labor reduction while maintaining crop yields and improving quality parameters. The economic analysis demonstrated system viability with 146% ROI and 0.69-year payback period for medium scale farms, addressing the need for field validated studies that quantify both agronomic and economic benefits.

The integrated framework represents a significant advancement over conventional precision agriculture systems by simultaneously addressing the intertwined challenges of data integrity, communication reliability, energy efficiency, and scalability issues previously treated in isolation. The seamless integration of edge intelligence with blockchain enables trust while meeting real-time latency requirements, filling the critical gap in multi-stakeholder agricultural environments.

Future work should focus on scaling the system to larger farm operations, integrating additional sensor types (nutrient levels, pest detection), and developing simplified deployment tools for smallholder farmers. The framework also shows potential for extension to other water-intensive crops in arid regions and integration with emerging technologies like AI-based predictive analytics and satellite remote sensing for enhanced decision support.

From a distributed computing perspective, this work demonstrates that CRT-based residue decomposition can transform sequential low-power communication into a parallel transmission problem with measurable speedup and predictable scaling behavior. It further establishes that permissioned ledger verification with sub-2-second end-to-end latency is feasible on ARM64 edge hardware, and that a layered

CAP-aware architecture can reconcile integrity guarantees with real-time actuation requirements in heterogeneous edge clusters. The framework’s energy-delay product and strong/weak scaling analyses provide quantitative evidence that number-theoretic methods can meaningfully enhance the performance of resource-constrained distributed systems. By validating these contributions in a demanding real-world deployment, we offer a blueprint for integrating parallel communication, edge consensus, and resilient actuation that extends beyond agriculture to any large-scale IoT environment requiring verifiable, low-latency control at the network edge.

Acknowledgments

We thank the participating farmers in Sfax for their collaboration and practical insights. We gratefully acknowledge the technical support provided by the University of Sfax IoT Research Group and the funding received from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Program. We also appreciate the data contributions from the Tunisia Meteorological Institute and the National Water Resources Authority.

References

- [1] Rathi, M., Gomathy, C.: Smart agriculture resource allocation and energy optimization using bi-lstm-aco. *Frontiers in Communications and Networks* **6** (2025)
- [2] Wang, L., Zhang, H., Liu, Y.: Energy-efficient lora-based communication for large-scale agricultural monitoring. *IEEE Internet of Things Journal* (2025)
- [3] Chen, Y., Li, X., Zhao, J.: Lightweight data compression techniques for resource-constrained iot edge devices. *IEEE Access* (2025)
- [4] Marcelloni, F., Vecchio, M.: A simple algorithm for data compression in wireless sensor networks. *IEEE Communications Letters* **12**(6), 411–413 (2009) <https://doi.org/10.1109/LCOMM.2008.080193>
- [5] Lin, J., Shen, Z., Zhang, A., Chai, Y.: Blockchain and iot based food traceability for smart agriculture. *IEEE Access* **8**, 543–552 (2020) <https://doi.org/10.1109/ACCESS.2019.2962487>
- [6] Ferrag, M.A., Shu, L., Debbah, M.: Blockchain technologies for the internet of things: Research issues and challenges. *IEEE Internet of Things Journal* **7**(10), 10288–10313 (2020) <https://doi.org/10.1109/JIOT.2020.2984478>
- [7] Tar, M., Hassan, U.: Hyperledger fabric-based decentralized data management framework for agricultural iot. *Sensors* **23**(2), 987 (2023)
- [8] Jayaram, S., Nanduri, R.: Hyperledger fabric for secure farm data management: Performance analysis and best practices. *Future Internet* **15**(2), 52 (2023)

- [9] Dube, A., colleagues: Performance of a hyperledger fabric-based smart irrigation system in arid farming. *Irrigation Science* **42**, 133–148 (2024)
- [10] Liu, Y., Yang, C., Jiang, L., Xie, S., Zhang, Y.: Intelligent edge computing for iot-based energy management in smart cities. *IEEE Network* **33**(2), 111–117 (2019) <https://doi.org/10.1109/MNET.2019.1800254>
- [11] Wang, X., Han, Y., Leung, V.C.M., Niyato, D., Yan, X., Chen, X.: Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* **22**(2), 869–904 (2021) <https://doi.org/10.1109/COMST.2020.2970550>
- [12] Jagan Mohan, S., Rayanoothala, R., Praneetha Sree, P.: Smart agriculture 5.0: Blockchain and reinforcement learning synergy for traceable iot-enabled supply chains. *Frontiers in Blockchain* (2026)
- [13] Tian, F.: An agri-food supply chain traceability system for china based on rfid and blockchain technology. 2016 13th International Conference on Service Systems and Service Management, 1–6 (2016) <https://doi.org/10.1109/ICSSSM.2016.7538424>
- [14] Kamilaris, A., Fonts, A., Prenafeta-Boldú, F.X.: The rise of blockchain technology in agriculture and food supply chains. *Trends in Food Science & Technology* **91**, 640–652 (2019) <https://doi.org/10.1016/j.tifs.2019.07.034>
- [15] Garner, H.L.: The residue number system. *IRE Transactions on Electronic Computers* **EC-8**(2), 140–147 (1959) <https://doi.org/10.1109/TEC.1959.5219515>
- [16] Ding, C., Pei, D., Salomaa, A.: Chinese Remainder Theorem: Applications in Computing, Coding, Cryptography. World Scientific Publishing, ??? (1996). <https://doi.org/10.1142/2893>
- [17] Wang, Y., Chen, I., Wang, D.: A novel data compression method for wireless sensor networks. *Sensors* **18**(4), 1057 (2018) <https://doi.org/10.3390/s18041057>
- [18] Huffman, D.A.: A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* **40**(9), 1098–1101 (1952)
- [19] Ziv, J., Lempel, A.: A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory* **23**(3), 337–343 (1977)
- [20] Shoup, V.: *A Computational Introduction to Number Theory and Algebra*, 2nd edn. Cambridge University Press, ??? (2009)
- [21] Knuth, D.E.: *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd edn. Addison-Wesley Professional, ??? (1997)

- [22] Mohammed, I., Yacoub, S.: Energy-efficient lora-based sensor platforms for precision agriculture: A 2025 review. *Computers and Electronics in Agriculture* **218**, 108920 (2025)
- [23] Raza, S., Wallgren, L., Voigt, T.: A comprehensive review of lorawan: From technology to security. *Sensors* **18**(11), 3995 (2018)
- [24] Brooks, T., Lin, A.: Precision water management in arid farming systems. *Agricultural Water Management* **223**, 105689 (2019)
- [25] Williams, J., Roberts, P.: Water savings in controlled irrigation systems supported by iot analytics. *Agricultural Water Management* **274**, 107942 (2022)
- [26] Miller, J., Torres, L.: Climate change impacts on irrigation demand in semi-arid regions. *Agricultural Water Management* **238**, 106238 (2020)
- [27] Wang, N., Zhang, N., Wang, M.: Wireless sensors in agriculture and food industry: Recent development and future perspective. *Computers and Electronics in Agriculture* **50**(1), 1–14 (2006) <https://doi.org/10.1016/j.compag.2005.09.003>
- [28] McBratney, A., Whelan, B., Ancev, T.: Future directions of precision agriculture. *Precision Agriculture* **6**, 7–23 (2005) <https://doi.org/10.1007/s11119-005-0681-8>
- [29] Zhang, N., Wang, M., Wang, N.: Precision agriculture: a worldwide overview. *Computers and Electronics in Agriculture* **36**(2), 113–132 (2002) [https://doi.org/10.1016/S0168-1699\(02\)00096-0](https://doi.org/10.1016/S0168-1699(02)00096-0)
- [30] Gebbers, R., Adamchuk, V.I.: Precision agriculture and food security. *Science* **327**(5967), 828–831 (2010) <https://doi.org/10.1126/science.1183899>
- [31] Bogaen, H.R., Huisman, J.A., Oberdörster, C., Vereecken, H.: Evaluation of a low-cost soil water content sensor for wireless network applications. *Journal of Hydrology* **344**(1), 32–42 (2010) <https://doi.org/10.1016/j.jhydrol.2007.06.032>
- [32] Pierce, F.J., Nowak, P.: Aspects of precision agriculture. *Advances in Agronomy* **67**, 1–85 (1999) [https://doi.org/10.1016/S0065-2113\(08\)60513-1](https://doi.org/10.1016/S0065-2113(08)60513-1)
- [33] Evans, R.G., Iversen, W.M., Kim, Y.: Integration of wireless sensor network with variable rate irrigation technology. *Proceedings of the ASABE Annual International Meeting* **12**, 1–13 (2008)
- [34] Aydin, G., Hall, M.J.: A comprehensive review of iot and wireless sensor networks in precision agriculture. *IEEE Access* **9**, 157733–157755 (2021) <https://doi.org/10.1109/ACCESS.2021.3129552>
- [35] Van Klompenburg, T., Kassahun, A., Catal, C.: Crop yield prediction using machine learning: A systematic literature review. *Computers and Electronics in*

- Agriculture **177**, 105709 (2020) <https://doi.org/10.1016/j.compag.2020.105709>
- [36] Mohanty, S.P., Hughes, D.P., Salathé, M.: Using deep learning for image-based plant disease detection. *Frontiers in Plant Science* **7**, 1419 (2016) <https://doi.org/10.3389/fpls.2016.01419>
 - [37] Goap, A., Sharma, D., Shukla, A.K., Rama Krishna, C.: An iot based smart irrigation management system using machine learning and open source technologies. *Computers and Electronics in Agriculture* **155**, 41–49 (2018) <https://doi.org/10.1016/j.compag.2018.09.040>
 - [38] Chlingaryan, A., Sukkarieh, S., Whelan, B.: Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review. *Computers and Electronics in Agriculture* **151**, 61–69 (2018) <https://doi.org/10.1016/j.compag.2018.05.012>
 - [39] Kamilaris, A., Fonts, A., Prenafeta-Boldó, F.X.: The rise of blockchain technology in agriculture and food supply chains. *Trends in Food Science & Technology* **91**, 640–652 (2019) <https://doi.org/10.1016/j.tifs.2019.07.034>
 - [40] Oh, S., Kim, H., Park, S.: Blockchain-based food safety management system for smart farm produce. *Food Control* **158**, 110245 (2025) <https://doi.org/10.1016/j.foodcont.2023.110245>
 - [41] Galvez, J.F., Mejuto, J., Simal-Gandara, J.: Future challenges on the use of blockchain for food traceability analysis. *TrAC Trends in Analytical Chemistry* **107**, 222–232 (2018) <https://doi.org/10.1016/j.trac.2018.08.011>
 - [42] Lin, J., Shen, Z., Zhang, A., Chai, Y.: Blockchain and iot based food traceability system for smart agriculture. In: *Proceedings of the 3rd International Conference on Crowd Science and Engineering*, pp. 1–6 (2018). <https://doi.org/10.1145/3265689.3265692>
 - [43] Ferrag, M.A., Shu, L., Yang, X., Derhab, A., Maglaras, L.: Security and privacy for green iot-based agriculture: Review, blockchain solutions, and challenges. *IEEE Access* **8**, 32031–32053 (2020) <https://doi.org/10.1109/ACCESS.2020.2973178>
 - [44] Haque, A.B., Bhushan, B., Dhiman, G.: Conceptualizing smart city applications: Requirements, architecture, security issues, and emerging trends. *Expert Systems* **41**(5), 13382 (2024) <https://doi.org/10.1111/exsy.13382>
 - [45] Rama, R., Kanchana, V., Sivasankari, N., Prabha, S.: A systematic review of blockchain technology for agriculture supply chain management. *Materials Today: Proceedings* **47**, 3148–3154 (2021) <https://doi.org/10.1016/j.matpr.2021.06.253>

- [46] Patil, A.S., Tama, B.A., Park, Y., Rhee, K.-H.: A framework for blockchain-based smart farming system using iot. *IEEE Access* **10**, 6845–6862 (2022) <https://doi.org/10.1109/ACCESS.2022.3142563>
- [47] Li, S., Da Xu, L., Zhao, S.: 5g internet of things: A survey. *Journal of Industrial Information Integration* **10**, 1–9 (2018) <https://doi.org/10.1016/j.jii.2018.01.005>
- [48] Kjellsson, J., Zakhori, A.: Adaptive sampling for sensor networks. *ACM Transactions on Sensor Networks* **4**(3), 1–39 (2008) <https://doi.org/10.1145/1362542.1362545>
- [49] Goldreich, O., Ron, D., Sudan, M.: Chinese remaindering with errors. In: *Proceedings of the 31st Annual ACM Symposium on Theory of Computing*, pp. 225–234 (1999). <https://doi.org/10.1145/301250.301308>
- [50] Sanchez, S., Blanc, S., Yuste, P., Serrano, J.J.: Energy-efficient data transmission in wireless sensor networks for precision agriculture. *Sensors* **21**(4), 1209 (2021) <https://doi.org/10.3390/s21041209>
- [51] Heinzelman, W.R., Chandrakasan, A., Balakrishnan, H.: Energy-efficient communication protocol for wireless microsensor networks. In: *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, pp. 10–20 (2000). <https://doi.org/10.1109/HICSS.2000.926982>
- [52] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., Watteyne, T.: Understanding the limits of lorawan. *IEEE Communications Magazine* **55**(9), 34–40 (2017) <https://doi.org/10.1109/MCOM.2017.1600613>
- [53] Li, S., Li, J., Liu, J., Wang, H., Zhao, S.: Performance analysis of lora networks for smart agriculture. *IEEE Internet of Things Journal* **7**(10), 10045–10057 (2020) <https://doi.org/10.1109/JIOT.2020.2999323>
- [54] Raj, E.R., Prasad, D.V.: A comprehensive study on lora technology for wireless sensor networks. *International Journal of Advanced Research in Computer Science* **8**(5), 1214–1219 (2017)
- [55] Akyildiz, I.F., Wang, X.: A survey on wireless mesh networks. *IEEE Communications Magazine* **43**(9), 23–30 (2005) <https://doi.org/10.1109/MCOM.2005.1509968>
- [56] Schmidt, A., Westenberger, M., Seitz, J., Kuladinithi, K., Timm-Giel, A.: B.a.t.m.a.n. for mobile ad-hoc networks. *Proceedings of IEEE CCNC*, 1–6 (2010) <https://doi.org/10.1109/CCNC.2010.5421784>
- [57] Chettri, L., Bera, R.: A comprehensive survey on internet of things (iot) toward 5g wireless systems. *IEEE Internet of Things Journal* **7**(1), 16–32 (2020) <https://doi.org/10.1109/JIOT.2019.2948888>

- [58] Sharma, S.K., Wang, X.: Live data analytics with collaborative edge and cloud processing in wireless iot networks. *IEEE Internet of Things Journal* **7**(7), 6071–6084 (2020) <https://doi.org/10.1109/JIOT.2020.2977767>
- [59] Parhami, B.: *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd edn. Oxford University Press, ??? (2010)
- [60] Espressif Systems: ESP32 Technical Reference Manual. (2020). Version 4.3. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [61] Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, pp. 483–485. ACM, ??? (1967). <https://doi.org/10.1145/1465482.1465560>
- [62] Inc., A.: Tdr-315n soil moisture sensor specifications. *Acclima Datasheet* (2022)
- [63] Electronics, A.: Dht22 temperature and humidity sensor datasheet. *AOSONG Technical Documentation* (2018)
- [64] Devices, A.: Low-power soil ph measurement techniques. *Analog Devices Application Note* (2013)
- [65] Components101: Bh1750 light sensor module. *Components101* (2020)
- [66] Byrne, M.: Calibrating the bh1750 for par measurement. *IotaWatt Engineering Blog* (2022)
- [67] Industries, A.: Rfm95w lora radio transceiver. *Adafruit Product Guide* (2016)
- [68] Foundation, R.P.: Raspberry pi 4 model b technical specifications. *Raspberry Pi Documentation* (2023)
- [69] Electronics, W.: Sx1302 lorawan gateway module overview. *Waveshare Wiki* (2024)
- [70] Corporation, M.: Trusted platform module (tpm) technology overview. *Microsoft Learn* (2023)
- [71] Network, T.T.: Eu868 lorawan frequency plan. *TTN Documentation* (2024)
- [72] Alliance, L.: Lorawan regional parameters v1.0. *LoRa Alliance Technical Specification* (2021)
- [73] Kishore, D.M.: Performance evaluation of mesh protocols in real-time test beds. *International Research Journal of Engineering and Technology* **2**(3), 2230–2236 (2015)

- [74] Hardy, G.H., Wright, E.M., Heath-Brown, R., Silverman, J.: An Introduction to the Theory of Numbers, 6th edn. Oxford University Press, ??? (2008)
- [75] Jay, C.: ESP32S3 Cache block diagram and operations. CSDN Blog. Accessed: 2025-10-03 (2025). https://blog.csdn.net/Cris_Jay/article/details/147540172
- [76] esp-rs Project: Cache Management and Memory Barriers: esp-hal Documentation. DeepWiki. Accessed: 2025-03-08 (2025). <https://deepwiki.com/esp-rs/esp-hal/8.3-cache-management-and-memory-barriers>
- [77] Community, S.O.: What if Dictionary size in LZW algorithm is full? Stack Overflow. Answer describing typical 4096-entry dictionary limit (2016). <https://stackoverflow.com/questions/40054218/what-if-dictionary-size-in-lzw-algorithm-is-full>
- [78] Semtech Corporation: LoRa Modem Designer’s Guide. (2015). AN1200.22 Rev 2
- [79] IBM: Hyperledger Fabric Overview. <https://www.ibm.com/docs/en/blockchain-platform> (2023)
- [80] Długosz, P., *et al.*: Hyperledger fabric access control system for iot layer in blockchain-based applications. *Sensors* **22**(20), 5981–5995 (2022) <https://doi.org/10.3390/s22205981>
- [81] Team, H.F.: Ordering Service Documentation. https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html (2024)
- [82] Gubbi, J., Buyya, R., Marusic, S., Palaniswami, M.: Internet of things (iot): A vision, architectural elements, and future directions. *Future generation computer systems* **29**(7), 1645–1660 (2013)
- [83] Christidis, K., Devetsikiotis, M.: Blockchains and smart contracts for the internet of things. *IEEE Access* **4**, 2292–2303 (2016)
- [84] Kaur, M.: Explained: Chaincode in Hyperledger Fabric. Medium. <https://medium.com/hyperledger/explained-chaincode-in-hyperledger-fabric-dcfb673e4071> (2020)
- [85] Team, H.F.: Private Data Collections Documentation. <https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html> (2024)
- [86] Seeds, J.S.: How to Grow Tomatoes from Seed. <https://www.johnnyseeds.com/growers-library/vegetables/tomatoes/tomatoes-key-growing-information.html>. Accessed 2025-11-16 (2024)
- [87] SeedCo, P.: Field Tomato Production Guide. <https://seedcogroup.com/bw/>

- [vegetables/wp-content/uploads/2021/12/Field-Tomato.pdf](#). Accessed 2025-11-16 (2021)
- [88] Solutions, I.G.: Fertilizers for Tomato. <https://icl-growingsolutions.com/en-us/agriculture/crops/tomato/>. Accessed 2025-11-16 (2023)
- [89] Program, U.A.S.I.P.M.: Fertilization: Tomato Pest Management Guidelines. <https://ipm.ucanr.edu/agriculture/tomato/fertilization/>. Accessed: 2025-11-16 (2024)
- [90] Missouri Extension, U.: Growing Home Garden Tomatoes. <https://extension.missouri.edu/publications/g6461>. Accessed 2025-11-16 (2020)
- [91] Extension, R.C.: Peppers — Mid-Atlantic Commercial Vegetable Production Recommendations 2024/2025. <https://njaes.rutgers.edu/pubs/commercial-veg-rec/peppers.pdf>. Accessed 2025-11-16 (2024)
- [92] Lindgren, D.T., Hodges, L., Browning, S.J.: Garden Peppers (NebGuide G1879). <https://extensionpubs.unl.edu/publication/g1879/2008/html/view>. Accessed 2025-11-16 (2008)
- [93] Shimray, A.G., *et al.*: Effect of spacing and nutrient management on growth and yield of king chilli (*capsicum chinense*). *Int. J. Curr. Microbiol. App. Sci.* **8**(8), 2761–2770 (2019). Provides recommended spacing and fertilizer doses for *Capsicum* spp.
- [94] Gonzalez, R., Horowitz, M.: Energy and delay: A metric for early design space exploration. *IEEE Micro* **20**(5), 56–64 (2000) <https://doi.org/10.1109/40.877952>
- [95] Ongaro, D., Ousterhout, J.: In search of an understandable consensus algorithm. *USENIX Annual Technical Conference*, 305–319 (2014)
- [96] Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 173–186 (1999)
- [97] Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 347–356 (2019). <https://doi.org/10.1145/3293611.3331591>
- [98] Bessani, A., Sousa, J., Alchieri, E.E.: State machine replication for the masses with bft-smart. *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 355–362 (2014) <https://doi.org/10.1109/DSN.2014.43>

- [99] Benali, A., Zribi, M.: Water scarcity trends in tunisia: Implications for agricultural policy. *Environmental Science & Policy* **158**, 45–59 (2024)
- [100] Adeloye, A., El-Mohtar, C.: Agricultural water allocation and efficiency in tunisia: A review of current trends. *Agricultural Systems* **192**, 103192 (2021)
- [101] Khan, R., Ali, S.: Enhancing blockchain throughput in iot systems using chinese remainder theorem-based parallelization. *IEEE Access* **9**, 113456–113470 (2021)
- [102] Djeddi, A., Karim, Y.: Crt-based blockchain transaction parallelization for improving throughput in iot. *IEEE Transactions on Network Science and Engineering* **12**(2), 221–234 (2025)
- [103] Yavari, M., Safkhani, M., Kumari, S., Kumar, S., Chen, C.-M.: An improved blockchain-based authentication protocol for iot network management. *Security and Communication Networks* **2020**, 1–16 (2020) <https://doi.org/10.1155/2020/8875217>
- [104] Kalaimathi, R., Yogeshwaran, A., Rajeswari, P.: Identifying false injection packet transmission using svm and secure olsr in wireless sensor networks. In: 4th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE), pp. 1–7 (2025). <https://doi.org/10.1109/ICDCECE65353.2025.11035330> . IEEE
- [105] Kumari, S., Singh, R., Kumar, A.: Building trust in iot networks: A rom-based key storage and cloud signature verification approach. In: 2025 IEEE International Conference on Internet of Things (iThings), pp. 1–8 (2025). <https://doi.org/10.1109/IOT.2025.11089136> . IEEE
- [106] Lu, Z., Wang, W., Zhang, X., Chen, L.: Reducing revocation latency in iot using edge computing and permissioned blockchain. In: 2022 IEEE International Conference on Blockchain (Blockchain), pp. 345–352 (2022). <https://doi.org/10.1109/Blockchain55522.2022.00055> . IEEE
- [107] Hyperledger Fabric Documentation: MSP Identity Validity Rules. <https://hyperledger-fabric.readthedocs.io/en/release-2.2/msp-identity-validity-rules.html>. Accessed: 2025-03-05 (2022)
- [108] Brewer, E.A.: Towards robust distributed systems. In: Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), p. 7 (2000). Invited talk
- [109] Gilbert, S., Lynch, N.: Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News* **33**(2), 51–59 (2002) <https://doi.org/10.1145/564585.564601>
- [110] Pritchett, D.: Base: An acid alternative. *Queue* **6**(3), 48–55 (2008) <https://doi.org/10.1145/1454111.1454112>

[org/10.1145/1394127.1394128](https://doi.org/10.1145/1394127.1394128)